【转载】第三讲——差分故障攻击的原理

转载 (https://www.qinless.com/category/reprint) · 2022-04-01 · 792 次浏览

文章目录[隐藏]

- 转载 20 / 25
- 说说在前面
- 一、前言
- 二、AES算法简述
- 三、密钥相关的三个问题
  - 3.1 获取了全体轮密钥和主密钥有区别吗
  - 3.2 可以从轮密钥逆推主密钥吗
  - 3.3 轮密钥所处的内存块可区分吗?
- 四、故障对密文的影响
  - 4.1 起始处发生故障对密文的影响
  - 4.2 倒数两个列混淆之间发生故障对密文的影响
  - 4.3 由密文状况反推故障发生地点
- 五、差分
- 六、工具选择
- 六、尾声

转载 (https://www.qinless.com/series/zhuanzai) 20 / 25

【转载】unidbg hook 使用大全 (https://www.qinless.com/670)

【转载】ida 基本使用 常用快捷键 (https://www.qinless.com/732)

【转载】Frida和调试器共存的问题 (https://www.qinless.com/937)

【转载】Charles抓包Android最佳流量转发APP (https://www.qinless.com/939)

【转载】安卓自定义代码调用系统函数并编译so记录 (https://www.qinless.com/1020)

【转载】jnitrace 手机卡死黑屏问题解决 (https://www.qinless.com/1169)

返回顶部

【转载】Unidbg SO 逆向入门实战教程一 OASIS (https://www.qinless.com/1173)

【转载】Unidbg SO 逆向入门实战教程二 calculateS (https://www.qinless.com/1177)

【转载】Unidbg SO 逆向入门实战教程三 V2-Sign (https://www.qinless.com/1181)

【转载】Unidbg SO 逆向入门实战教程四 mfw (https://www.qinless.com/1185)

【转载】Unidbg SO 逆向入门实战教程五 qxs (https://www.qinless.com/1188)

【转载】Unidbg SO 逆向入门实战教程六 s (https://www.qinless.com/1192)

【转载】Unidbg SO 逆向入门实战教程七 main (https://www.qinless.com/1194)

【转载】Unidbg SO 逆向入门实战教程八 文件读写 (https://www.qinless.com/1201)

【转载】Unidbg SO 逆向入门实战教程九 blackbox (https://www.qinless.com/1204)

【转载】Unidbg SO 逆向入门实战教程十 simpleSign (https://www.qinless.com/1209)

【转载】android app 加密参数分析研究 hash aes (https://www.qinless.com/1634)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型 (https://www.qinless.com/1642)

【转载】第二讲——白盒加密攻击方法的选择 (https://www.qinless.com/1645)

【转载】第三讲——差分故障攻击的原理

【转载】第四讲——差分故障攻击的工具链 (https://www.qinless.com/1653)

【转载】第五讲——使用源码进行DFA攻击 (https://www.qinless.com/1655)

【转载】第六讲——使用Frida进行DFA攻击 (https://www.qinless.com/1657)

【转载】第七讲——使用Unidbg进行DFA攻击 (https://www.qinless.com/1659)

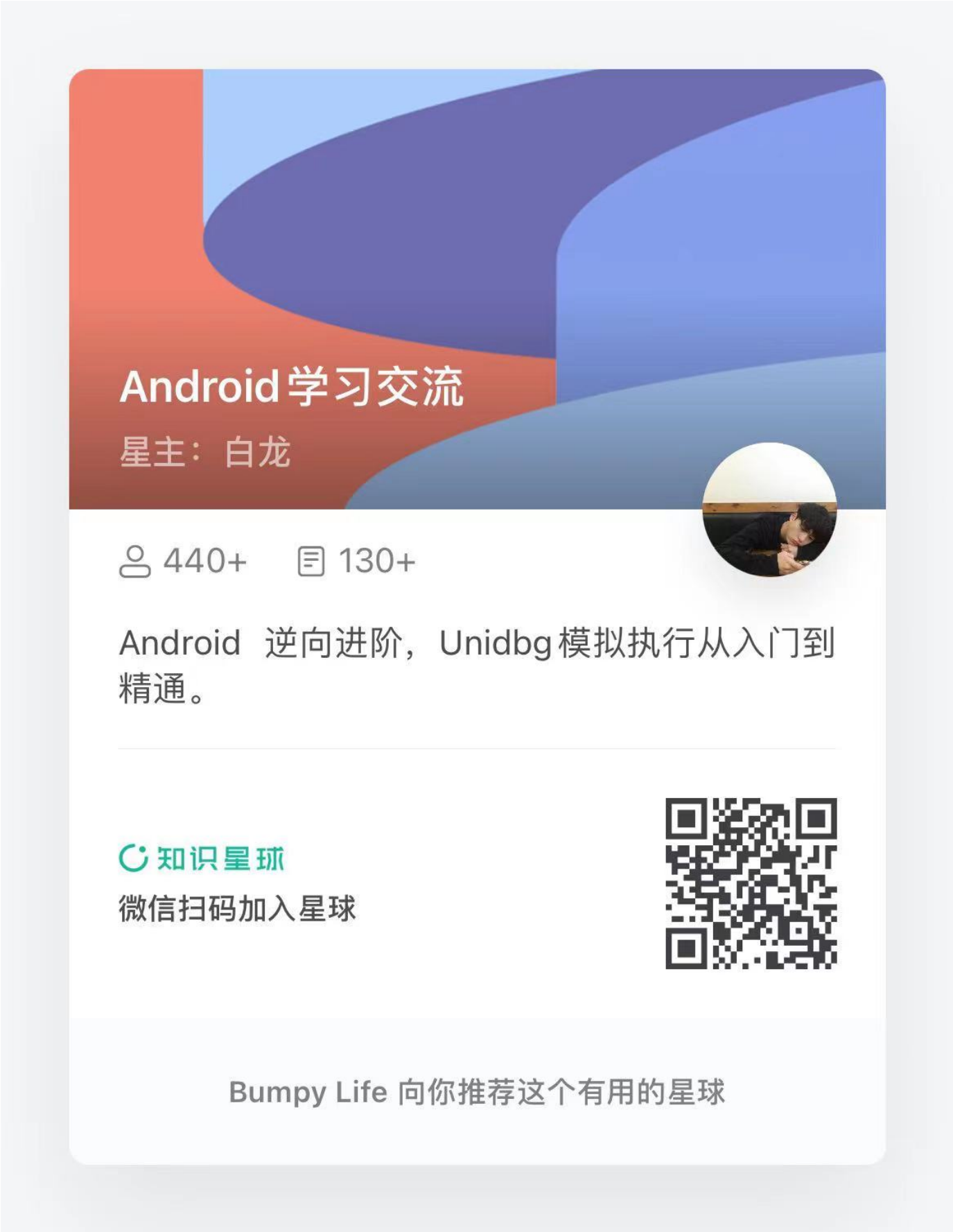
【转载】第八讲——案例 (https://www.qinless.com/1661)

« 上一篇文章 (https://www.qinless.com/1645)
下一篇文章 (https://www.qinless.com/1653) »

说说在前面

本文转载龙哥星球 白盒 AES 密码学系列，相关文件代码等，可加入星球下载

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/03/longge-zsxq.jpeg)

返回顶部

一、前言

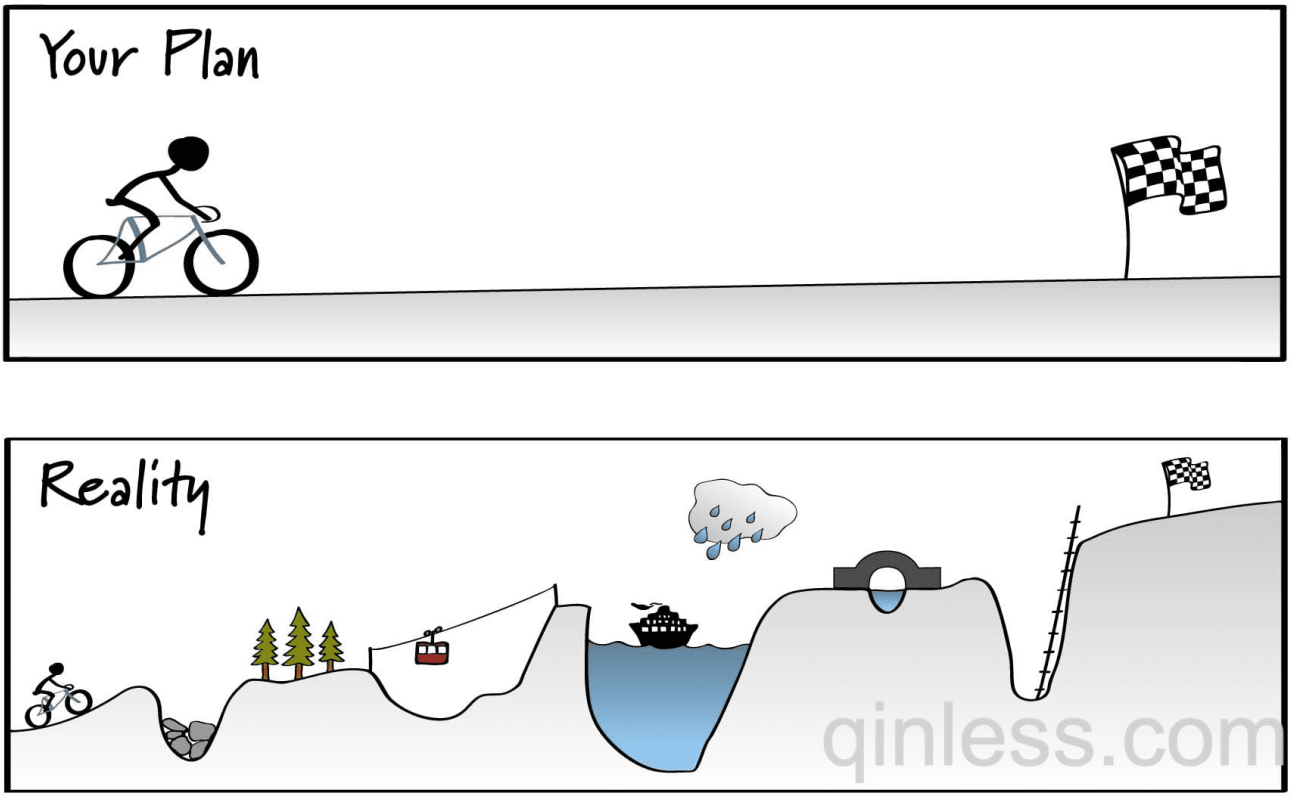
Differential Fault Analysis 中文翻译即差分错误分析或差分故障分析，Differential 即差分，Fault 为错误、故障之意。简而言之，本文要理解这个方法，就要探究这两个要素

- 差分
- 错误，或者说故障

这节的知识量可真不小，不妨在这里做一下简述。

在第二节中，简述了AES的加密流程，因为本节以及后续系列文章都围绕着AES转，我们希望和读者在AES上有共同且基础的理解。AES有加密和解密流程，也有128/192/256三个密钥规格。我们侧重于AES-128的加密流程。其余部分知识，整体类似，建议读者自行学习。文字+图的形式进行讲解，在代码方面，开源项目tiny-AES-C是非常标准的AES实现，但它是用C写的，为了尽可能减少读者的阅读门槛，我们用自写的Python代码。

本文会尽量如下情况，降低学习曲线以及避免深坑。



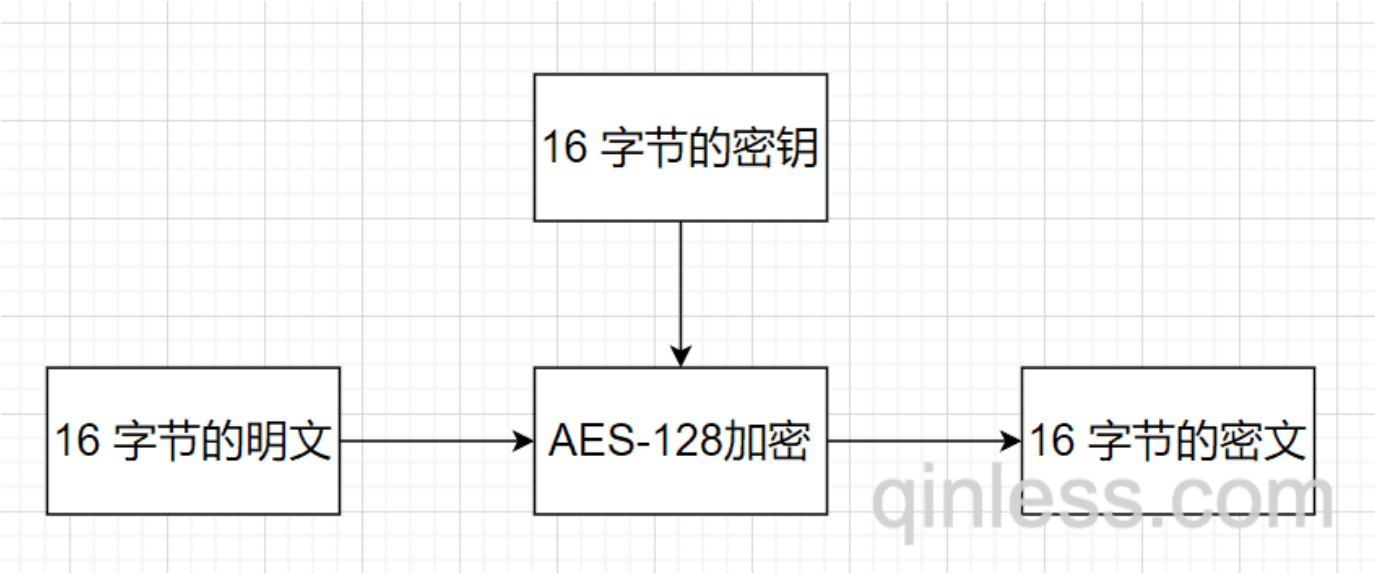
(https://www.qinless.com/wp-content/uploads/2022/04/6246ff6aea8a7.png)

二、AES算法简述

返回顶部

首先简单讨论AES算法本身，对AES真正熟悉的读者可以跳过这部分。工作模式以及填充方式并非AES独有的内容，这里不做讨论。

AES-128接收16字节的明文输入，16字节的密钥，输出16字节的密文结果。



(https://www.qinless.com/wp-content/uploads/2022/04/6246ff717b98c.png)

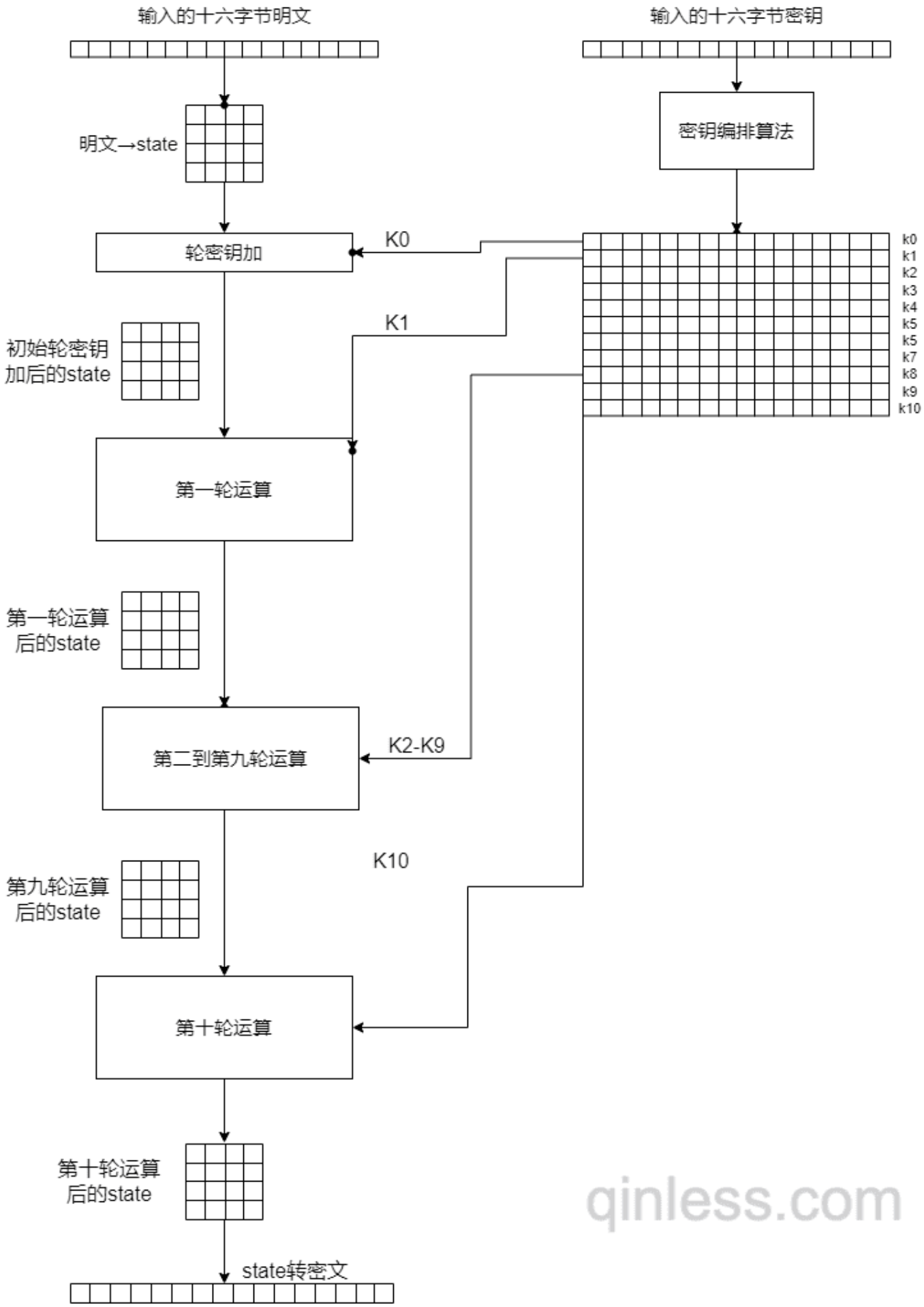
我们key设置为**2b7e151628aed2a6abf7158809cf4f3c**，明文设置为**00112233445566778899aabbccddeeff**。

```
input_bytes = 0x00112233445566778899aabbccddeeff
key = 0x2b7e151628aed2a6abf7158809cf4f3c
```

COPY

接下来进入AES的世界,下方是整体流程图

返回顶部



qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6246ff7843362.png)

返回顶部

首先看整体的流程图，我们发现，AES的整体图景可以分成左右两块，即明文的处理和密钥的编排。明文的处理主体是一个初始化轮密钥加和十轮运算，在初始化轮密钥加十轮运算中都需要使用密钥编排的结果。密钥编排将16个字节经过运算推演出11组轮密钥，每一组16个字节，称之为 $K_0, K_1 .... K_{10}$

下面我们看一下密钥扩展是如何算出来的，这是我们的密钥Key **2b7e151628aed2a6abf7158809cf4f3c**。为了区分密钥和密钥编排后的轮密钥，我们将此时的密钥叫主密钥。

在AES-128中，密钥扩展后得16*11共176字节，使用时**逐十六个字节**划分成 $K_0, K_1, ... K_{10}$ 使用，但是在生成时，它是**逐四个字**节生成的，即 44*\4。我们不妨用数组来描述它，即一个包含了44个元素的数组,叫 W 。

这四十四个元素的生成规则有三种，如下图所示

返回顶部

w0	w1	w2	w3	K0
w4	w5	w6	w7	K1
w8	w9	w10	w11	K2
w12	w13	w14	w15	K3
w16	w17	w18	w19	K4
w20	w21	w22	w23	K5
w24	w25	w26	w27	K6
w28	w29	w30	w31	K7
w32	w33	w34	w35	K8
w36	w37	w38	w39	K9
w40	w41	w42	w43	K10

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6246ff802c833.png)

不同颜色代表了不同规则。最上方的 W_0, W_1, W_2, W_3 就是主密钥本身切成四段。

$$Key = 2b7e151628aed2a6abf7158809cf4f3c$$
$$W_0 = 2b7e1516$$
$$W_1 = 28aed2a6$$
$$W_2 = abf71588$$
$$W_3 = 09cf4f3c$$

返回顶部

左侧的红色部分， $W_4, W_8, W_{12}, ...W_{40}$ 的生成复杂一点。

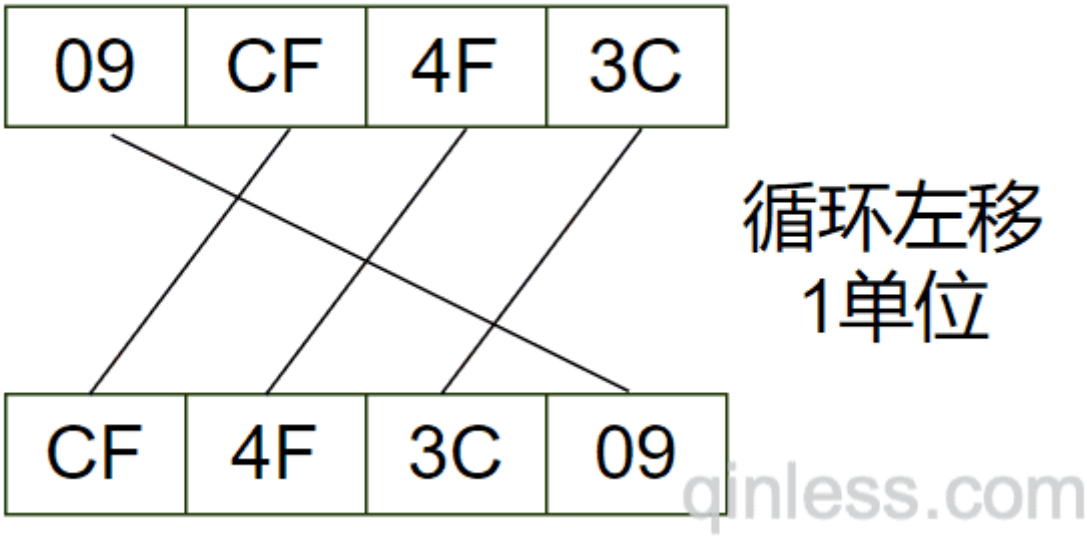
$$W_n = g(W_{n-1}) \quad xor \quad W_{n-4}$$

xor 是异或运算，比如 $W_4 = g(W_3) \quad xor \quad W_0$ 。g(当前元素前面那个元素) 异或 当前元素头顶上那个元素。

那么关键点就是这个 g 函数了， g 函数一共三个步骤——循环左移、S盒替换、字节异或。我们以 $W4$ 运算中所需的 W_3 为例。

$$W_3 = 09cf4f3c$$

首先是循环左移，规则固定——将最左边的一个字节挪到右边即可



(https://www.qinless.com/wp-content/uploads/2022/04/6246ff86d9665.png)

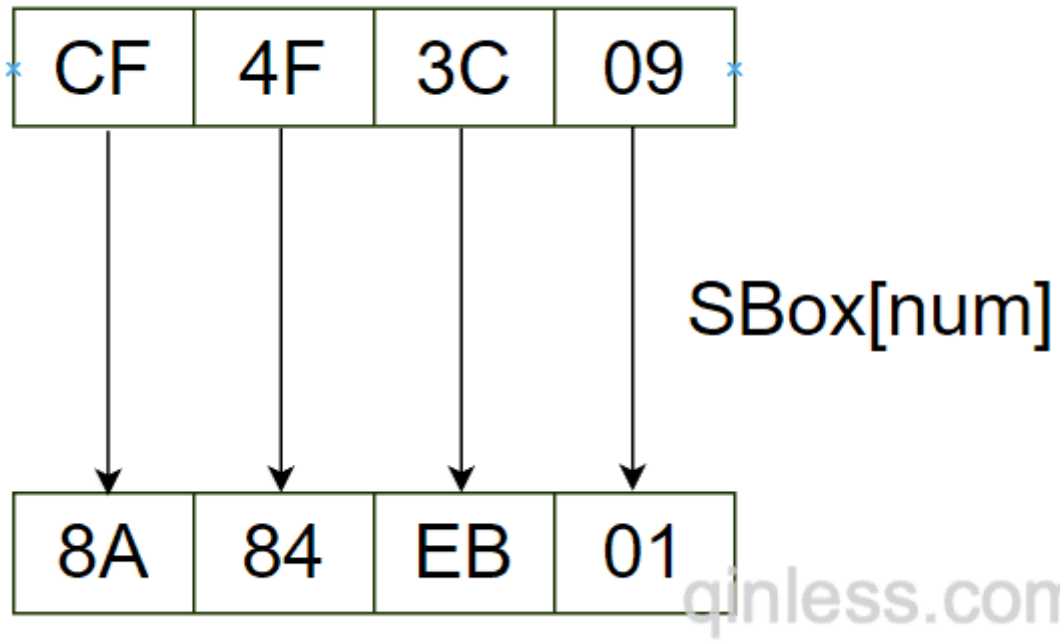
第二步是S盒替换，S盒替换听着很高级，但操作上很简单——将数值本身作为索引取出S数组中对用的值。S盒是固定的，在Findcrypt中，就会利用S盒来查找AES的存在。

返回顶部

```
SBox = [
0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16
]

num = 0x13
result = SBox[num]
## 7d
```

S盒的背后有十分复杂的知识，但好在我们并不需要去了解。

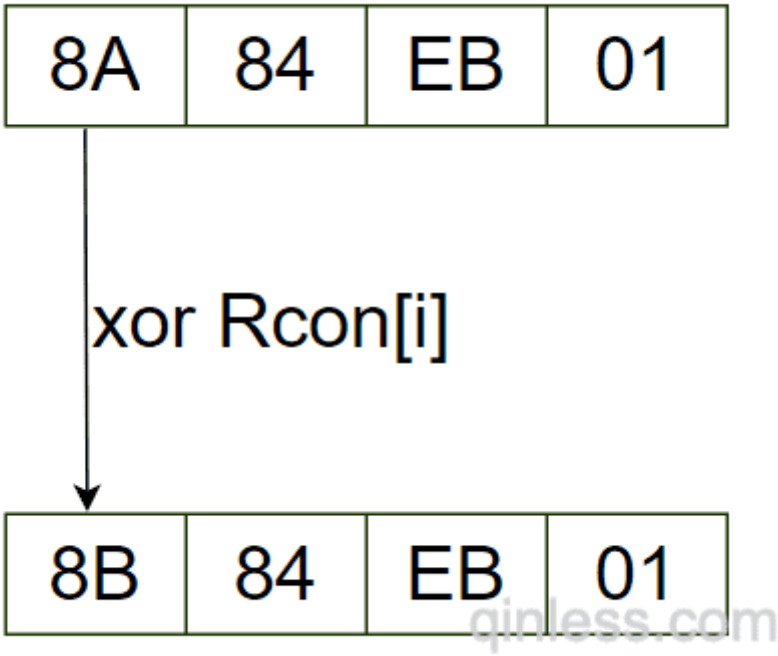


返回顶部

(https://www.qinless.com/wp-content/uploads/2022/04/6246ff8eec922.png)

最后一个步骤更简单，将上一步结果中的最高字节和一个固定常量异或。W4的生成是第一个，用如下rcon表的第一个元素0x1。W40即第十次，用最后一个元素0x36.

```
rcon = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36]
```



(https://www.qinless.com/wp-content/uploads/2022/04/6246ff941e82c.png)

$$
\begin{aligned}
W_4 &= g(W_3) \quad xor \quad W_1 \\
&= g(09cf4f3c) \quad xor \quad 2b7e1516 \\
&= 8b84eb01 \quad xor \quad 2b7e1516 \\
&= a0fafe17
\end{aligned}
$$

最后一步的异或我用Python算的

```
print(hex(0x8b84eb01 ^ 0x2b7e1516))
```

上图中蓝色和红色的部分我们都讲完了，那么橙色部分呢？相当的简单，和红色部分类似，去掉g函数即可

$$W_n = W_{n-1} \quad xor \quad W_{n-4}$$

返回顶部

打个比方， $W_5 = W_4 \quad xor \quad W_1 = 0xa0fafe17 \quad xor \quad 0x28aed2a6 = 0x88542cb1$

如下是完整的密钥编排部分的Python代码

返回顶部

COPY

```
Sbox = (
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
)

Rcon = (0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36)

def text2matrix(text):
    matrix = []
    for i in range(16):
        byte = (text >> (8 * (15 - i))) & 0xFF
        if i % 4 == 0:
            matrix.append([byte])
        else:
            matrix[i // 4].append(byte)
    return matrix

def shiftRound(array, num):
    '''

    :param array: 需要循环左移的数组
    :param num: 循环左移的位数
    :return: 使用Python切片，返回循环左移num个单位的array
    '''
    return array[num:] + array[:num]

def g(array, index):
    '''
    g 函数
```

返回顶部

```
    :param array: 待处理的四字节数组
    :index:从1-10，每次使用Rcon中不同的数
    '''
    # 首先循环左移1位
    array = shiftRound(array, 1)
    # 字节替换
    array = [Sbox[i] for i in array]
    # 首字节和rcon中对应元素异或
    array = [(Rcon[index] ^ array[0])] + array[1:]
    return array

def xorTwoArray(array1, array2):
    '''
    返回两个数组逐元素异或的新数组
    :param array1: 一个array
    :param array2: 另一个array
    :return:
    '''
    assert len(array1) == len(array2)
    return [array1[i] ^ array2[i] for i in range(len(array1))]

def showRoundKeys(kList):
    for i in range(len(kList)):
        print("K%02d:" %i +"".join("%02x" % k for k in kList[i]))

def keyExpand(key):
    master_key = text2matrix(key)
    round_keys = [[0] * 4 for i in range(44)]
    # 规则一(图中红色部分)
    for i in range(4):
        round_keys[i] = master_key[i]
    for i in range(4, 4 * 11):
        # 规则二(图中红色部分)
        if i % 4 == 0:
            round_keys[i] = xorTwoArray(g(round_keys[i - 1], i // 4), round_keys[i - 4])
        # 规则三(图中橙色部分)
        else:
            round_keys[i] = xorTwoArray(round_keys[i - 1], round_keys[i - 4])
    # 将轮密钥从44*4转成11*16,方便后面在明文的运算里使用
    kList = [[] for i in range(11)]
    for i in range(len(round_keys)):
        kList[i//4] += round_keys[i]
```

返回顶部

```
    showRoundKeys(kList)
    return kList

input_bytes = 0x00112233445566778899aabbccddeeff
key = 0x2b7e151628aed2a6abf7158809cf4f3c
kList = keyExpand(key)
```

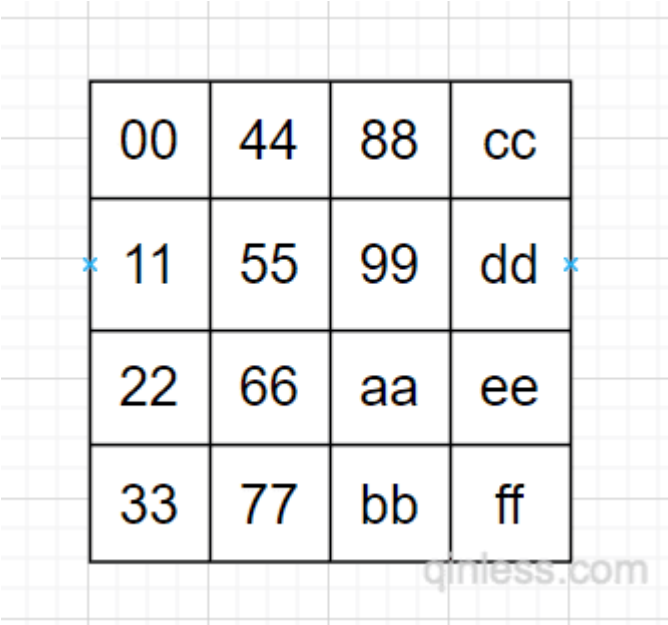
运行结果如下

```
K00:2b7e151628aed2a6abf7158809cf4f3c
K01:a0fafe1788542cb123a339392a6c7605
K02:f2c295f27a96b9435935807a7359f67f
K03:3d80477d4716fe3e1e237e446d7a883b
K04:ef44a541a8525b7fb671253bdb0bad00
K05:d4d1c6f87c839d87caf2b8bc11f915bc
K06:6d88a37a110b3efddbf98641ca0093fd
K07:4e54f70e5f5fc9f384a64fb24ea6dc4f
K08:ead27321b58dbad2312bf5607f8d292f
K09:ac7766f319fadc2128d12941575c006e
K10:d014f9a8c9ee2589e13f0cc8b6630ca6
```

我们对AES密钥编排部分的学习就基本完成了，在下一小节中会讨论三个密钥相关的问题。现在开始学习明文的运算，即图中左边的部分。首先，我们要调整明文的格式，在AES中，数据以 $state$ 的形式计算、中间存储和传输，中文名即**状态**。

从明文转到state形式很简单，以我们的明文**00112233445566778899aabbccddeeff**为例。从上到下，从左到右。千万不要颠倒顺序，第一行不是“00 11 22 33”。除此之外，**state中的数，我们一般用十六进制表示，且不加0x前缀，这样看着比较舒服。除非特意强调是十进制，否则下文均为十六进制。**

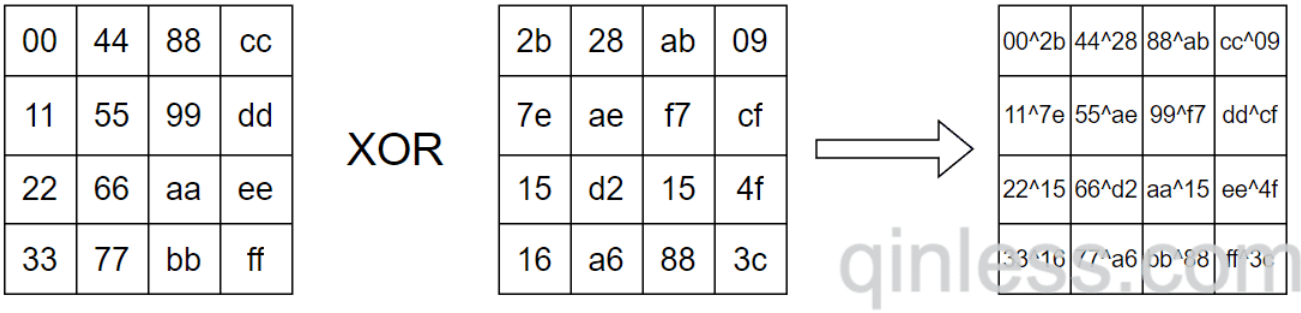
返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246ff9fb9ea4.png)

接下来是轮密钥加步骤，因为是第一次轮密钥加步骤，所以也叫初始轮密钥加。轮密钥加步骤听着很怪，但实质很简单。只需要将对应的轮密钥和 $state$ 一样从上到下，从左到右排列。两个矩阵逐字节异或，这就是轮密钥加步骤。为什么要叫轮密钥加而不是轮密钥异或？我们卖个关子，后面再说。

初始的轮密钥加使用 K_0 ，2b7e151628aed2a6abf7158809cf4f3c。



(https://www.qinless.com/wp-content/uploads/2022/04/6246ffa53c028.png)

接下来就是十轮主运算，看如下的伪代码，我们可以清楚看到一轮运算中有什么，以及第十轮和前九轮有什么区别。

返回顶部

$$
\begin{aligned}
&\text{state} \leftarrow \text{plaintext} \\
&\text{AddRoundKey}\ (\text{state}, k_0) \\
&\text{for } r = 1 \ldots 9 \\
&\quad \text{SubBytes(state)} \\
&\quad \text{ShiftRows(state)} \\
&\quad \text{MixColumns(state)} \\
&\quad \text{AddRoundKey(state}, k_r) \\
&\text{SubBytes(state)} \\
&\text{ShiftRows(state)} \\
&\text{AddRoundKey(state}, k_{10}) \\
&\text{ciphertext} \leftarrow \text{state}
\end{aligned}
$$

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6246ffaab3eae.png)

初始的明文转 $state$ 和最后的 $state$ 转明文自不必说，然后是初始轮密钥，使用 K_0 。

前九轮运算中，包含四个步骤：字节替换，循环左移，列混淆，轮密钥加。

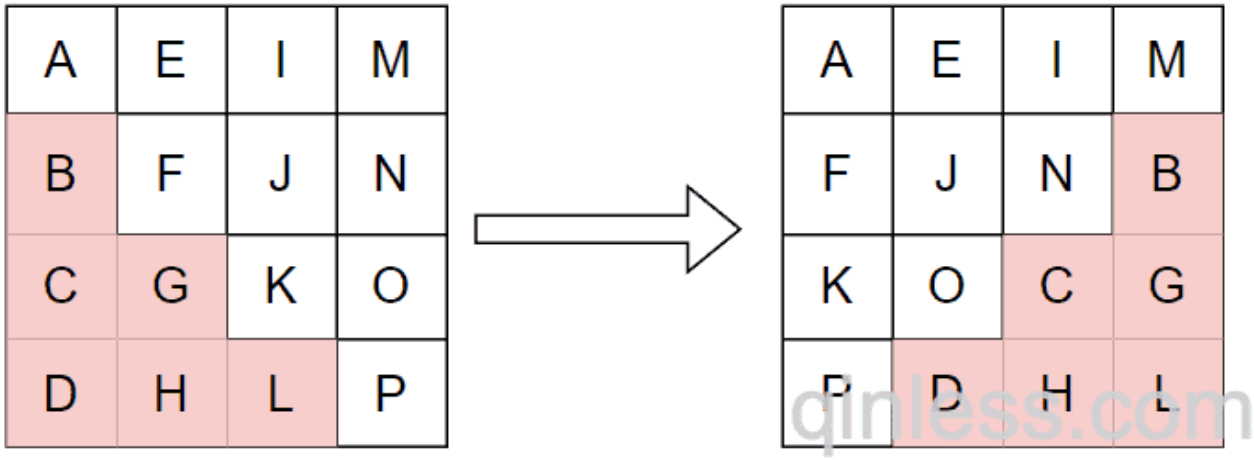
第十轮中，包含三个步骤：字节替换，循环左移，轮密钥加。相比前九轮缺一个列混淆，其余相同。

十轮运算中的轮密钥加，和初始轮密钥加相比，除了使用的轮密钥不同外，并无不同，分别为 $K_1.....K_{10}$ 。

而 $SubBytes$ 字节替换步骤，和密钥编排中的S盒替换完全一致。

$ShiftRows$ 即循环左移，和密钥编排中的循环左移类似，但有差异。密钥编排中， g 函数中也需循环左移，但其中待处理的数据仅有一行，而明文编排中 $state$ 是四行。其循环左移规则如下：第一行不循环左移，第二行循环左移1字节，第三行循环左移2字节，第四行循环左移3字节。

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246ffb00fdf8.png)

相对复杂的是列混淆步骤，列混淆步骤涉及两块知识，1是矩阵乘法，2是伽罗瓦域的加法和乘法。前者还好，后者属于抽象代数的内容，比较复杂。我并不打算阐述其中的数理知识，以其折磨我的读者，因为我也非常讨厌数学，所以我们只服用最小剂量的相关知识。

先看第一块——矩阵乘法。首先演示简单的矩阵相乘，

$$\begin{Bmatrix} 1 & 2 \\ 4 & 5 \end{Bmatrix} * \begin{Bmatrix} 7 & 4 \\ 2 & 6 \end{Bmatrix} = \begin{Bmatrix} a & b \\ c & d \end{Bmatrix}$$

左边准备相乘的两个矩阵，我们称它俩为矩阵A和矩阵B，如何求结果矩阵中的 $abcd$ ？规则如下：第m行第n列的值等于**矩阵**A的第m行的元素与**矩阵**B的第n列对应元素乘积之和。

a是第一行第一列，那么就是A的第一行和B的第一列元素**乘积之和**

$$a = 1 * 7 + 2 * 2 = 11$$

同理可得

$$\begin{aligned} b &= 1 * 4 + 2 * 6 = 16 \\ c &= 4 * 7 + 5 * 2 = 38 \\ d &= 4 * 4 + 5 * 6 = 46 \end{aligned}$$

即

返回顶部

$$\begin{Bmatrix} 1 & 2 \\ 4 & 5 \end{Bmatrix} * \begin{Bmatrix} 7 & 4 \\ 2 & 6 \end{Bmatrix} = \begin{Bmatrix} 11 & 16 \\ 38 & 46 \end{Bmatrix}$$

所谓乘积之和，指乘法和加法。

再来看AES列混淆中的矩阵乘法，我们的 $state$ ，左边乘如下所示固定矩阵

$$\begin{Bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{Bmatrix} * \begin{Bmatrix} A & E & I & M \\ B & F & J & N \\ C & G & K & O \\ D & H & L & P \end{Bmatrix}$$

看起来有些复杂，小例子中是2*2的矩阵要算4个值，这里是4*4的矩阵要算16个值。我们这里只管第一列，其他列的计算类似。

$$\begin{Bmatrix} 2A+3B+C+D & ... & ... & ... \\ A+2B+3C+D & ... & ... & ... \\ A+B+2C+3D & ... & ... & ... \\ 3A+B+C+2D & ... & ... & ... \end{Bmatrix}$$

列混淆中的的加法和乘法并不是小例子或日常中的那样，其中的加法指**异或运算**。2A + 3B + C + D 即 2A ^ 3B ^ C ^ D，这也是 $AddRoundKeys$ 叫轮密钥加而不是轮密钥异或的原因——加法就是异或。那么其中的乘法呢？乘法复杂一些，想真正理解的读者可以网上冲浪搜索**伽罗瓦域内乘法**。我们这里仅考虑如下三种情况，因为AES-128加密中，列混淆的乘法中，仅涉及这三个数。

$$x * 1 = ?$$
$$x * 2 = ?$$
$$x * 3 = ?$$

结合Python代码可以更清晰，函数名中 mul 是multiply（乘）的缩写。

返回顶部

x * 1，结果为x本身

```
def mul_by_01(num):
    return num
```

x * 2，首先切换到二进制形式，最高位为0时，比特串左移1比特，最右边补0即可。如果最高位为1，比特串左移1比特，最右边补0，最后再异或上 0x1B

```
def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b

    return res % 0x100
```

x *3 = (x* 02) + x，注意”加“是异或哦。

```
def mul_by_03(num):
    return (mul_by_02(num) ^ num)
```

列混淆，就这么讲完了。下面看完整的AES代码实现

返回顶部

COPY

```
Sbox = (
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
)

Rcon = (0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36)

def text2matrix(text):
    matrix = []
    for i in range(16):
        byte = (text >> (8 * (15 - i))) & 0xFF
        if i % 4 == 0:
            matrix.append([byte])
        else:
            matrix[i // 4].append(byte)
    return matrix

def shiftRound(array, num):
    '''

    :param array: 需要循环左移的数组
    :param num: 循环左移的位数
    :return: 使用Python切片，返回循环左移num个单位的array
    '''
    return array[num:] + array[:num]

def g(array, index):
    '''
    g 函数
```

返回顶部

```
    :param array: 待处理的四字节数组
    :index:从1-10，每次使用Rcon中不同的数
    '''
    # 首先循环左移1位
    array = shiftRound(array, 1)
    # 字节替换
    array = [Sbox[i] for i in array]
    # 首字节和rcon中对应元素异或
    array = [(Rcon[index] ^ array[0])] + array[1:]
    return array

def xorTwoArray(array1, array2):
    '''
    返回两个数组逐元素异或的新数组
    :param array1: 一个array
    :param array2: 另一个array
    :return:
    '''
    assert len(array1) == len(array2)
    return [array1[i] ^ array2[i] for i in range(len(array1))]

def showRoundKeys(round_keys):
    # 将轮密钥从44*4转成11*16
    kList = [[] for i in range(11)]
    for i in range(len(round_keys)):
        kList[i // 4] += round_keys[i]
    for i in range(len(kList)):
        print("K%02d:" % i + "".join("%02x" % k for k in kList[i]))

def keyExpand(key):
    master_key = text2matrix(key)
    round_keys = [[0] * 4 for i in range(44)]
    # 规则一(图中红色部分)
    for i in range(4):
        round_keys[i] = master_key[i]
    for i in range(4, 4 * 11):
        # 规则二(图中红色部分)
        if i % 4 == 0:
            round_keys[i] = xorTwoArray(g(round_keys[i - 1], i // 4), round_keys[i - 4])
        # 规则三(图中橙色部分)
        else:
            round_keys[i] = xorTwoArray(round_keys[i - 1], round_keys[i - 4])
    showRoundKeys(round_keys)
```

返回顶部

```
    return round_keys

def AddRoundKeys(state, roundKey):
    result = [[] for i in range(4)]
    for i in range(4):
        result[i] = xorTwoArray(state[i], roundKey[i])

    return result

def SubBytes(state):
    result = [[] for i in range(4)]
    for i in range(4):
        result[i] = [Sbox[i] for i in state[i]]
    return result

def ShiftRows(s):
    s[0][1], s[1][1], s[2][1], s[3][1] = s[1][1], s[2][1], s[3][1], s[0][1]
    s[0][2], s[1][2], s[2][2], s[3][2] = s[2][2], s[3][2], s[0][2], s[1][2]
    s[0][3], s[1][3], s[2][3], s[3][3] = s[3][3], s[0][3], s[1][3], s[2][3]
    return s

def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b
    return res % 0x100

def mul_by_03(num):
    return mul_by_02(num) ^ num

def MixColumns(state):
    for i in range(4):
        s0 = mul_by_02(state[i][0]) ^ mul_by_03(state[i][1]) ^ state[i][2] ^ state[i][3]
        s1 = state[i][0] ^ mul_by_02(state[i][1]) ^ mul_by_03(state[i][2]) ^ state[i][3]
        s2 = state[i][0] ^ state[i][1] ^ mul_by_02(state[i][2]) ^ mul_by_03(state[i][3])
        s3 = mul_by_03(state[i][0]) ^ state[i][1] ^ state[i][2] ^ mul_by_02(state[i][3])
        state[i][0] = s0
        state[i][1] = s1
        state[i][2] = s2
        state[i][3] = s3
```

返回顶部

```
    return state
```

```
def state2Text(state):
    text = sum(state, [])
    return "".join("%02x" % k for k in text)

def encrypt(input_bytes, kList):
    '''

    :param input_bytes: 输入的明文
    :param kList: K0-K10
    :return:
    '''
    plainState = text2matrix(input_bytes)
    # 初始轮密钥加
    state = AddRoundKeys(plainState, kList[0:4])
    for i in range(1, 10):
        state = SubBytes(state)
        state = ShiftRows(state)
        state = MixColumns(state)
        state = AddRoundKeys(state, kList[4 * i:4 * (i + 1)])

    state = SubBytes(state)
    state = ShiftRows(state)
    state = AddRoundKeys(state, kList[40:44])
    return state

input_bytes = 0x00112233445566778899aabbccddeeff
key = 0x2b7e151628aed2a6abf7158809cf4f3c
kList = keyExpand(key)
cipherState = encrypt(input_bytes, kList)
cipher = state2Text(cipherState)
print(cipher)
```

因为代码没封装，没面向对象，所以接近于伪代码，非常好理解。

三、密钥相关的三个问题

这一节中，我们讨论关于密钥编排的，更深的一些问题。我们将密钥本身叫主密钥，编排后的叫扩展密钥、轮密钥、子密钥，我们采用轮密钥这个叫法。本节部分问题与主题无关，但我觉得很有意思，所以还是写了进来。

这四页部

3.1 获取了全体轮密钥和主密钥有区别吗

回想一下上面的Python AES实现，在逻辑上，主密钥经过密钥编排后，生成11个轮密钥，用于具体运算。那么假如我们不知道主密钥内容，仅知道轮密钥，那么可以程序稍作修改，传入全体轮密钥，无需密钥编排，直接用于具体轮运算。

那么读者可能会有两个疑问

- AES 第一个轮密钥 K_0 就是主密钥本身，怎么可能知道全体轮密钥，但又不知道主密钥?
- 什么场景下会只获得了全体轮密钥的值，而不知道主密钥，这个需求点在哪儿呢?

先看第一个问题，AES 确实如此，但我们讨论的不仅是AES，其余包含密钥编排的加密算法，比如DES等，并不存在这种编排规则。

再讨论第二个问题，主要是两个场景，我们这里只说一个，还有一个场景在下一小节。AES/DES等算法的整体可分为密钥的编排和明文的运算，假设魔改了密钥的编排部分，主密钥在被密钥编排后得到的轮密钥结果和标准密钥不同。那么如果

- 主密钥基本不变，甚至固定
- 密钥编排算法魔改的程度较高，分析和还原比较复杂

这就适合直接用编排后的全体轮密钥做运算。

3.2 可以从轮密钥逆推主密钥吗

首先讨论AES-128

返回顶部

```
K00:2b7e151628aed2a6abf7158809cf4f3c
K01:a0fafe1788542cb123a339392a6c7605
K02:f2c295f27a96b9435935807a7359f67f
K03:3d80477d4716fe3e1e237e446d7a883b
K04:ef44a541a8525b7fb671253bdb0bad00
K05:d4d1c6f87c839d87caf2b8bc11f915bc
K06:6d88a37a110b3efddbf98641ca0093fd
K07:4e54f70e5f5fc9f384a64fb24ea6dc4f
K08:ead27321b58dbad2312bf5607f8d292f
K09:ac7766f319fadc2128d12941575c006e
K10:d014f9a8c9ee2589e13f0cc8b6630ca6
```

假如获取到的是 K_0 ，自不必说。如果获取的是 K_{10} 呢?

d014f9a8c9ee2589e13f0cc8b6630ca6，首先我们会到 W 数组的视图，看 K_{10} 密钥怎么编排出来的。

返回顶部

w0	w1	w2	w3	K0
w4	w5	w6	w7	K1
w8	w9	w10	w11	K2
w12	w13	w14	w15	K3
w16	w17	w18	w19	K4
w20	w21	w22	w23	K5
w24	w25	w26	w27	K6
w28	w29	w30	w31	K7
w32	w33	w34	w35	K8
w36	w37	w38	w39	K9
w40	w41	w42	w43	K10

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6246ffcb380a5.png)

K10是 $W_{40}, W_{41}, W_{42}, W_{43}$ 拼起来的，我们已知 K_{10} ，即已知 $W_{40}, W_{41}, W_{42}, W_{43}$ 。有没有办法求 K_9 ？如果可以，那么同理可以逆推到 K_0 ，即求得了主密钥。

$$W_{40} = g(W_{39}) \quad xor \quad W_{36}$$
$$W_{41} = W_{40} \quad xor \quad W_{37}$$
$$W_{42} = W_{41} \quad xor \quad W_{38}$$
$$W_{43} = W_{42} \quad xor \quad W_{39}$$

返回顶部

我们不妨先复习一下异或的基本性质，**异或作用与比特位上，对应的比特位相同则为0，相异则为1。**

$$
\begin{aligned}
0 \quad xor \quad 0 &= 0 \\
1 \quad xor \quad 0 &= 1 \\
0 \quad xor \quad 1 &= 1 \\
1 \quad xor \quad 1 &= 0
\end{aligned}
$$

因为相同为0，相异为1，那么一个数和自身异或时，因为每个比特位都相同，所以结果为0。

$$X + X = 0$$

而当某个数和0异或时，自身为0的比特0^0得0，自身为1的比特位1^0得1，这就导致结果和没异或前一样。

$$X + 0 = X$$

除此之外，异或并不看谁先谁后，A ^ B 与 B ^ A 显然无区别，即具有交换律。

$$X + Y = Y + X$$

矩阵乘法不具有交换律，大家还记得吗，矩阵结果的第m行第n列的值等于**矩阵**A的第m行的元素与**矩阵**B的第n列对应元素乘积之和。谁在前谁在后影响了用行还是用列。

学累的可以刷会儿知乎，看看异或的多角度理解：如何理解「异或」的含义(https://www.zhihu.com/question/31116687)。

下面做变换，左右和 W_{42} 异或

$$
\begin{aligned}
W_{43} &= W_{42} \quad xor \quad W_{39} \\
W_{42} \quad xor \quad W_{43} &= W_{42} \quad xor \quad W_{42} \quad xor \quad W_{39} \\
W_{42} \quad xor \quad W_{43} &= 0 \quad xor \quad W_{39} \\
W_{39} &= W_{42} \quad xor \quad W_{43}
\end{aligned}
$$

返回顶部

K_{10} 中涉及到的四个式子均可以做变化

$$
\begin{aligned}
W_{36} &= g(W_{39}) \quad xor \quad W_{40} \\
W_{37} &= W_{40} \quad xor \quad W_{41} \\
W_{38} &= W_{41} \quad xor \quad W_{42} \\
W_{39} &= W_{42} \quad xor \quad W_{43}
\end{aligned}
$$

K_{10} = d014f9a8c9ee2589e13f0cc8b6630ca6，切成四块

$$
\begin{aligned}
W_{40} &= d014f9a8 \\
W_{41} &= c9ee2589 \\
W_{42} &= e13f0cc8 \\
W_{43} &= b6630ca6
\end{aligned}
$$

```
>>> hex(0xd014f9a8^0xc9ee2589)
'0x19fadc21'
>>> hex(0xc9ee2589^0xe13f0cc8)
'0x28d12941'
>>> hex(0xe13f0cc8^0xb6630ca6)
'0x575c006e'
```

求出了 W_{37} 、 W_{38} 、 W_{39} ，即 K_9 的后大半部分，和真实情况比对后发现一致。

K09:ac7766f3**19fadc2128d12941575c006e**

那么 W_{36} 呢？再复习一下 g 函数吧

首先循环左移一字节，575c006e 变成 5c006e57

然后逐字节S盒替换，得4a639f5b

返回顶部

```
>>> Sbox = (
...     0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xA
...     0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x7
...     0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x3
...     0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB
...     0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2
...     0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x5
...     0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9
...     0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF
...     0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x1
...     0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0
...     0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE
...     0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xA
...     0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8
...     0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1
...     0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x2
...     0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xB
... )
>>> hex(Sbox[0x5c])
'0x4a'
>>> hex(Sbox[0x00])
'0x63'
>>> hex(Sbox[0x6e])
'0x9f'
>>> hex(Sbox[0x57])
'0x5b'
```

最后是首字节和Rcon中的一个字节异或，这是最后一次变换，即用0x36

```
Rcon = (0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36)
```

```
>>> hex(0x4a^0x36)
'0x7c'
```

g函数的结果即7c639f5b

```
>>> hex(0x7c639f5b ^ 0xd014f9a8)
'0xac7766f3'
```

返回顶部

完整的K9: **ac7766f319fadc2128d12941575c006e**，就被我们分析出来了。可以继续往上推出K0，获得主密钥。即AES可以依靠轮密钥逆推出主密钥。严肃的说，AES-128可以通过一轮轮密钥逆推出主密钥，AES-192需要一轮半，AES-256需要两轮轮密钥。感兴趣的读者可以去处理一下AES-192/256。

接下来讨论另外两个情况

1是DES，DES密钥长八字节，在密钥编排时，会舍弃每个字节的最后1比特。因此实际参与密钥编排的是56比特，我们可以验证这一点。

假设密钥是

```
11 22 33 44 55 66 77 88 (hex)
```

COPY

二进制即

0001000**1** 0010001**0** 0011001**1** 0100010**0** 0101010**1** 0110011**0** 0111011**1** 1000100**0**

我们将末尾1比特翻转，即0变1，1变0。如果AES密钥中每个字节的最后一位被“丢弃”，那么翻转后也并不会对程序有任何影响，反之加密结果应该不同。

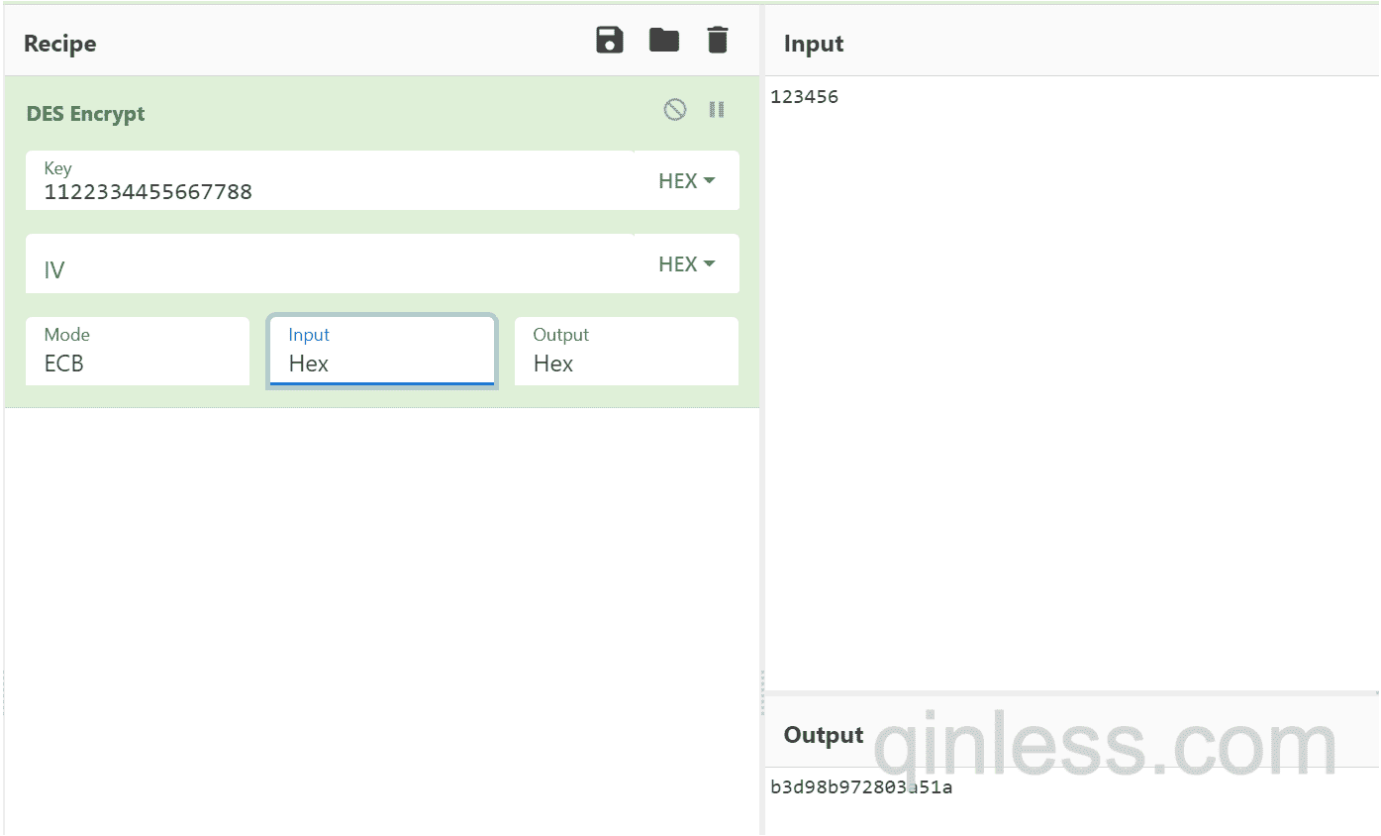
0001000**0** 0010001**1** 0011001**0** 0100010**1** 0101010**0** 0110011**1** 0111011**0** 1000100**1**

转成十六进制即

```
10 23 32 45 54 67 76 89 (hex)
```

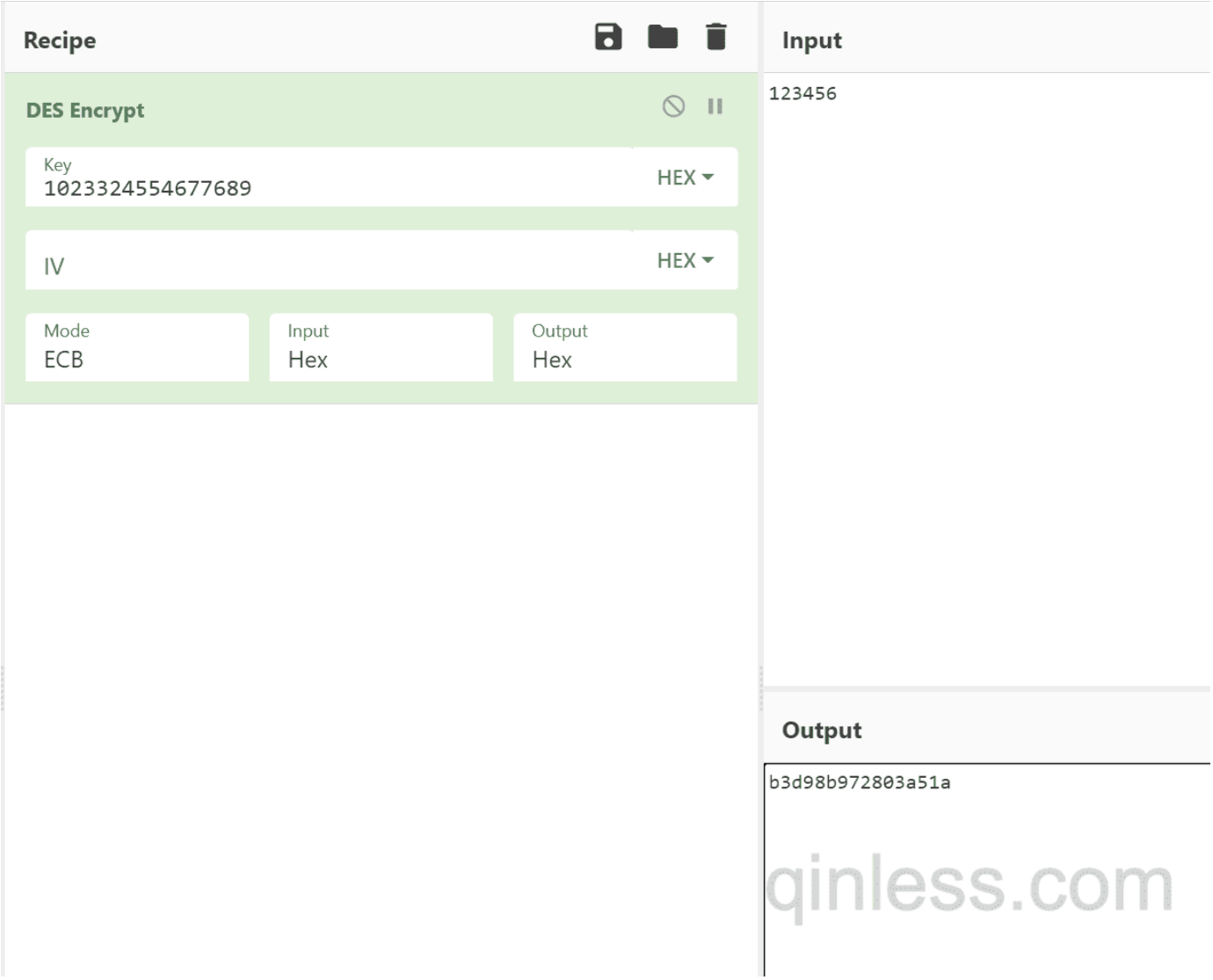
在Cyberchef中，使用这两个密钥进行DES加密，结果完全一致。

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246ffd679620.png)

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246ffdbd1315.png)

对于DES而言，已知**两轮或两轮**以上的**子密钥**，就可逆推出初始密钥，具体代码见 DES 子密钥逆推初始密钥 (https://www.freebuf.com/column/219723.html) 一文。对于主密钥中无法复原的八比特，随便补什么都行，反正不影响结果。

在实际场景中，当我们需要基于 AES/DES 的轮密钥恢复主密钥时，我们用 SideChannelMarvels/Stark (https://github.com/SideChannelMarvels/Stark) 这个开源项目，用户体验很好。

在另外一些情况中，目标加密算法可以基于某个轮密钥逆推出全体轮密钥，但没法恢复出主密钥，那怎么办呢——这不就是3.1吗？

3.3 轮密钥所处的内存块可区分吗？

首先得考虑一个代码编写的问题，**在大多数加密实现中，出于模块化以及效率的考量，密钥编排作为一个整体性的、前置的步骤，在明文的运算前就被完全算出来。**以AES -128为例，Key 为 16字节，密钥编排后生成11个16字节的轮密钥，这是我们刚学的。很少有程序会在明文运算中，用

返回顶部

到了 K_i 再去编排生成那16字节，而是像我们的Python代码那样，提前生成所有轮密钥，供后续明文运算时使用。在Android Native开发中而言，即K0-K10紧凑的在一块内存中，装在比如 `uint8_t RoundKey[176]` 这样的数组里。我们将全体轮密钥称为轮密钥群，这么叫比较好听。

那么如果提供给我们一串字节数据，能否判断其为AES的轮密钥群？比如如下的176字节

```
66 f4 f0 bf   7e 2c c6 76   30 ec c1 51   15 06 e2 cc   | f•••~,•v0••Q•••• |
08 6c bb e6   76 40 7d 90   46 ac bc c1   53 aa 5e 0d   | •l••v@}•F•••S•^• |
a6 34 6c 0b   d0 74 11 9b   96 d8 ad 5a   c5 72 f3 57   | •4l••t•••••Z•r•W |
e2 39 37 ad   32 4d 26 36   a4 95 8b 6c   61 e7 78 3b   | •97•2M&6•••la•x; |
7e 85 d5 42   4c c8 f3 74   e8 5d 78 18   89 ba 00 23   | ~••BL••t•]x••••# |
9a e6 f3 e5   d6 2e 00 91   3e 73 78 89   b7 c9 78 aa   | •••••.••>sx•••x• |
67 5a 5f 4c   b1 74 5f dd   8f 07 27 54   38 ce 5f fe   | gZ_L•t_•••'T8•_• |
ac 95 e4 4b   1d e1 bb 96   92 e6 9c c2   aa 28 c3 3c   | •••K••••••••••(•< |
18 bb 0f e7   05 5a b4 71   97 bc 28 b3   3d 94 eb 8f   | •••••Z•q••(•=••• |
21 52 7c c0   24 08 c8 b1   b3 b4 e0 02   8e 20 0b 8d   | !R|•$••••••••  •• |
a0 79 21 d9   84 71 e9 68   37 c5 09 6a   b9 e5 02 e7   | •y!••q•h7••j•••• |
```

这个问题有什么意义？

一个比较小的作用在于，在重度混淆的样本里，我们没法自上而下的分析逻辑。那么如果发现某个参数指向这么一片内存，我们可能需要办法确认——这是不是编排后的轮密钥？

返回顶部

w0	w1	w2	w3	K0
w4	w5	w6	w7	K1
w8	w9	w10	w11	K2
w12	w13	w14	w15	K3
w16	w17	w18	w19	K4
w20	w21	w22	w23	K5
w24	w25	w26	w27	K6
w28	w29	w30	w31	K7
w32	w33	w34	w35	K8
w36	w37	w38	w39	K9
w40	w41	w42	w43	K10

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6246ffe72e407.png)

比较取巧的办法是验证图中橙色部分，验证w5是否等于w4^w1，w6是否等于w5^w2等等。

那么比较体系化的办法呢？那就是结合3.2，我们假定上面字节流的最后十六个字节是轮密钥最后一轮，然后逆推出整个轮密钥群，两相对比，如果一致就可确认是否是轮密钥群。

在stark中进行验证，传入的参数中，**a07921d98471e96837c5096ab9e502e7** 是假设的轮密钥（字节流最后十六字节），10为第几轮。

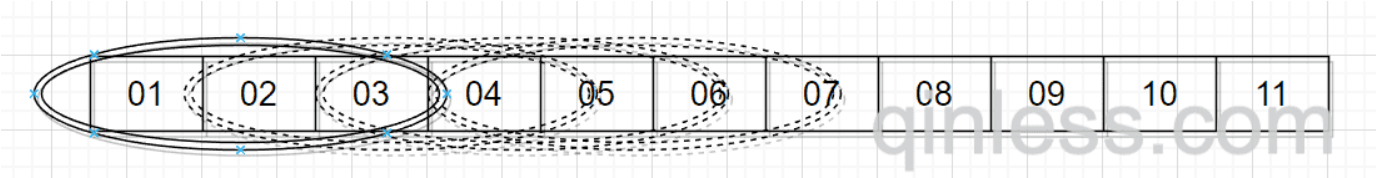
返回顶部

```
C:\Users\13352\CLionProjects\mystark\cmake-build-debug>mystark.exe a07921d98471e96837c5096ab9e5
K00: 66F4F0BF7E2CC67630ECC1511506E2CC
K01: 086CBBE676407D9046ACBCC153AA5E0D
K02: A6346C0BD074119B96D8AD5AC572F357
K03: E23937AD324D2636A4958B6C61E7783B
K04: 7E85D5424CC8F374E85D781889BA0023
K05: 9AE6F3E5D62E00913E737889B7C978AA
K06: 675A5F4CB1745FDD8F07275438CE5FFE
K07: AC95E44B1DE1BB9692E69CC2AA28C33C
K08: 18BB0FE7055AB47197BC28B33D94EB8F
K09: 21527CC02408C8B1B3B4E0028E200B8D
K10: A07921D98471E96837C5096AB9E502E7
```

比对后可确认一致。AES中还有一个特殊之处，第一轮子密钥就是主密钥，所以密钥就是66F4F0BF7E2CC67630ECC1511506E2CC。又因为AES“**第一轮子密钥就是主密钥**“，所以有个更简单和易懂的办法——假设可疑内存块的前十六个字节是主密钥，做密钥编排，对比两者结果是否一致。

在实践中，要考虑三个问题

一是几百M或更大的内存中，我们并不能提前得知哪176字节是“可疑的”，所以得逐字节从前往后推移，校验接下来的176字节是否符合密钥编排的规则。图示中将176字节缩减成3字节，展示逐步往后推移的过程。



(https://www.qinless.com/wp-content/uploads/2022/04/6246ffed4146f.png)

在某些处理器和系统中，读写内存需四字节对齐，那么第二/三/四字节就不可能是密钥编排结果的起始地址，这种情况下可以逐四字节验证，节省不少时间。

二是一个符合密钥编排规则的内存块，就一定是密钥编排的结果吗？会不会误判？

如果是12345这样的短字节串，内存中恰好出现的可能性不小，但176个字节很长，恰好符合AES的密钥编排规律，这概览很小。打个比方，3.14可能表达了其他含义而非圆周率，而**3.14159265358979323846264338327950288419716939937 51**极大概率就是表示圆周率，而不是其他含义。

返回顶部

三是工程实践中，存在大小端的问题

上文给出的176字节，是Tiny-AES-C 实现中，密钥编排的结果

```
66 f4 f0 bf   7e 2c c6 76   30 ec c1 51   15 06 e2 cc   | f•••~,•v0••Q•••• |
08 6c bb e6   76 40 7d 90   46 ac bc c1   53 aa 5e 0d   | •l••v@}•F•••S•^• |
a6 34 6c 0b   d0 74 11 9b   96 d8 ad 5a   c5 72 f3 57   | •4l••t•••••Z•r•W |
e2 39 37 ad   32 4d 26 36   a4 95 8b 6c   61 e7 78 3b   | •97•2M&6•••la•x; |
7e 85 d5 42   4c c8 f3 74   e8 5d 78 18   89 ba 00 23   | ~••BL••t•]x••••# |
9a e6 f3 e5   d6 2e 00 91   3e 73 78 89   b7 c9 78 aa   | •••••.••>sx•••x• |
67 5a 5f 4c   b1 74 5f dd   8f 07 27 54   38 ce 5f fe   | gZ_L•t_•••'T8•_• |
ac 95 e4 4b   1d e1 bb 96   92 e6 9c c2   aa 28 c3 3c   | •••K••••••••••(•< |
18 bb 0f e7   05 5a b4 71   97 bc 28 b3   3d 94 eb 8f   | •••••Z•q••(•=••••|
21 52 7c c0   24 08 c8 b1   b3 b4 e0 02   8e 20 0b 8d   | !R|•$••••••••••••|
a0 79 21 d9   84 71 e9 68   37 c5 09 6a   b9 e5 02 e7   | •y!••q•h7••j••••|
```

下面是Openssl中相同密钥在编排后的结果

```
bf f0 f4 66   76 c6 2c 7e   51 c1 ec 30   cc e2 06 15   | •••fv•,~Q••0••••|
e6 bb 6c 08   90 7d 40 76   c1 bc ac 46   0d 5e aa 53   | ••l••}@v•••F•^•S |
0b 6c 34 a6   9b 11 74 d0   5a ad d8 96   57 f3 72 c5   | •l4•••t•Z•••W•r•|
ad 37 39 e2   36 26 4d 32   6c 8b 95 a4   3b 78 e7 61   | •79•6&M2l•••;x•a |
42 d5 85 7e   74 f3 c8 4c   18 78 5d e8   23 00 ba 89   | B••~t••L•x]•#•••|
e5 f3 e6 9a   91 00 2e d6   89 78 73 3e   aa 78 c9 b7   | ••••••.••xs>•x••|
4c 5f 5a 67   dd 5f 74 b1   54 27 07 8f   fe 5f ce 38   | L_Zg•_t•T'•••_•8 |
4b e4 95 ac   96 bb e1 1d   c2 9c e6 92   3c c3 28 aa   | K••••••••••••<•(•|
e7 0f bb 18   71 b4 5a 05   b3 28 bc 97   8f eb 94 3d   | ••••q•Z••(•••••= |
c0 7c 52 21   b1 c8 08 24   02 e0 b4 b3   8d 0b 20 8e   | •|R!•••$••••••••|
d9 21 79 a0   68 e9 71 84   6a 09 c5 37   e7 02 e5 b9   | •!y•h•q•j••7••••|
```

比较两者，后者每四字节相对于前者是逆序。这是因为处理逻辑上不太一样，前者作为字节数组处理，后者作为int处理。

如果用处理前者的方式处理后者，假定d92179a068e971846a09c537e702e5b9是最后一轮的轮密钥，使用stark

返回顶部

```
C:\Users\13352\CLionProjects\mystark\cmake-build-debug>mystark.exe d92179a068e971846a09c537e702
K00: AC073AE3EA8DE4925AED6B50A65C2F05
K01: E71251C70D9FB5555772DE05F12EF100
K02: D4B33266D92C87338E5E59367F70A836
K03: 817137B4585DB087D603E9B1A9734187
K04: 06F220675EAF90E088AC795121DF38D6
K05: 88F5D69AD65A467A5EF63F2B7F2907FD
K06: 0D308248DB6AC432859CFB19FAB5FCE4
K07: 9880EB6543EA2F57C676D44E3CC328AA
K08: 36B4478E755E68D9B328BC978FEB943D
K09: C49660FDB1C8082402E0B4B38D0B208E
K10: D92179A068E971846A09C537E702E5B9
```

那么，我们会得出这串字节并非密钥编排结果的错误结论。所以实践中，我们还会将每四字节进行反转，进行尝试。

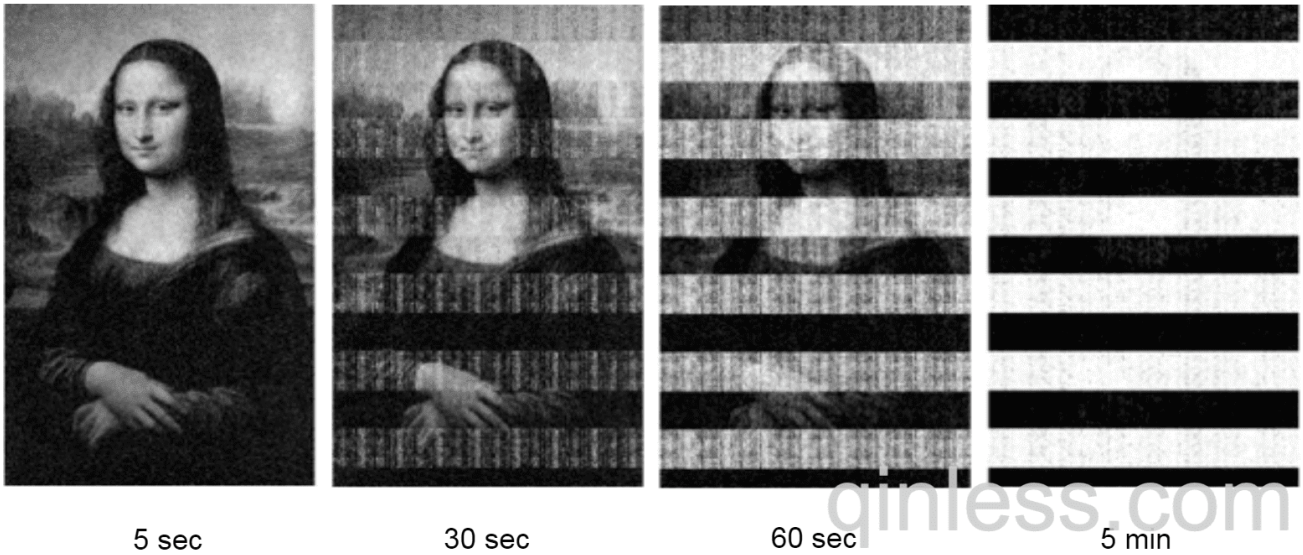
这三个问题都处理后，我们就可以像模像样写一个FindAesKey或者FindXXXKey函数。它接收一个大的字节数组，然后以单线程或并发的方式去遍历检索AES的Key。可是这个“大的字节数组”或者说内存数据哪里来的呢？

在白盒攻击场景下，我们可以直接访问目标进程的内存。在Android上，不管是ADB，还是IDA/Frida这样的工具，都可以实现内存的dump。

在黑盒场景下，可以扣出设备中的存储器硬件，然后用引导程序拷贝出内存镜像。但存在一个问题，物理方式拆卸存储器的过程中，设备往往需要断电。断电存在两个现象，存储器的数据不会**立刻**消失，这叫做**数据留存**。但在很短的时间内会迅速丢失，这叫做**数据衰退**。

下图是内存中的蒙娜丽莎图像，断电后，数据一开始很完整，在很短的时间里，数据就丢失了大半。

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246fffc81f06.png)

几秒或几十秒的时间，不足以在工程实践中完整拷贝出内存镜像，这极大的影响了方法的实操性。好在研究人员发现，降低设备温度，可以放缓数据衰退的过程，如存储器放置在-50°C的环境中，断电十分钟后，内存中数据的衰退率不超过1%。如下是放在液氮罐里的存储器。

返回顶部

返回顶部

(https://www.qinless.com/wp-content/uploads/2022/04/624700012d909.png)

这种攻击被称为冷启动攻击。

但不管是白盒场景下的dump内存还是灰盒场景下的冷启动攻击，都存在一个问题——只获取了某一瞬时的内存情况。这个时刻一定和加密相关吗？

从逻辑上说，我们可以控制dump时机晚于加密发生，比如先执行“登录”等操作，然后dump，这些敏感操作中肯定存在加密步骤。

但是加密结束后，可以也可能会清除、打乱密钥编排数据所处的内存块。除此之外，即使并非有意为之，内存也可能被重复利用，导致数据被覆盖。时机过早会导致加密还没开始，过晚可能已被清除或覆盖。

尽管有这样的缺点，这个方法还是有用的，至少也是可用方法之一。mmozeiko的 aes-finder (https://github.com/mmozeiko/aes-finder) 项目就是白盒场景下这个思路的知名开源实现。

而在Unidbg这一强大的模拟执行工具上，可以做到更强大的FindKey，这归功于两方面

- Unidbg作为一个小型模拟器，程序的运行空间相比真实系统小得多，对某一时机的全体内存做FindKey，所需时间极少。
- Unidbg可以逐汇编/基本块/函数做拦截Hook，那么可以选择在这些结束后遍历内存，进行FindKey。如果逐行汇编或逐个基本块结束后检索，其时间成本要数小时，如果仅在函数结束后检索，那么一般项目不超过10分钟就可完成，而且可以据此定位到密钥产生的第一时机——密钥编排函数或至少是AES加密的内部逻辑。我的**Unidbg_FindKey (https://github.com/Pr0214/Unidbg_FindKey)** 就是基于此设计的代码逻辑。

即因为内存小，以及拦截的能力，所以可以进行多次内存采样，增强FindKey的功能，以及避免时机的遗漏。

四、故障对密文的影响

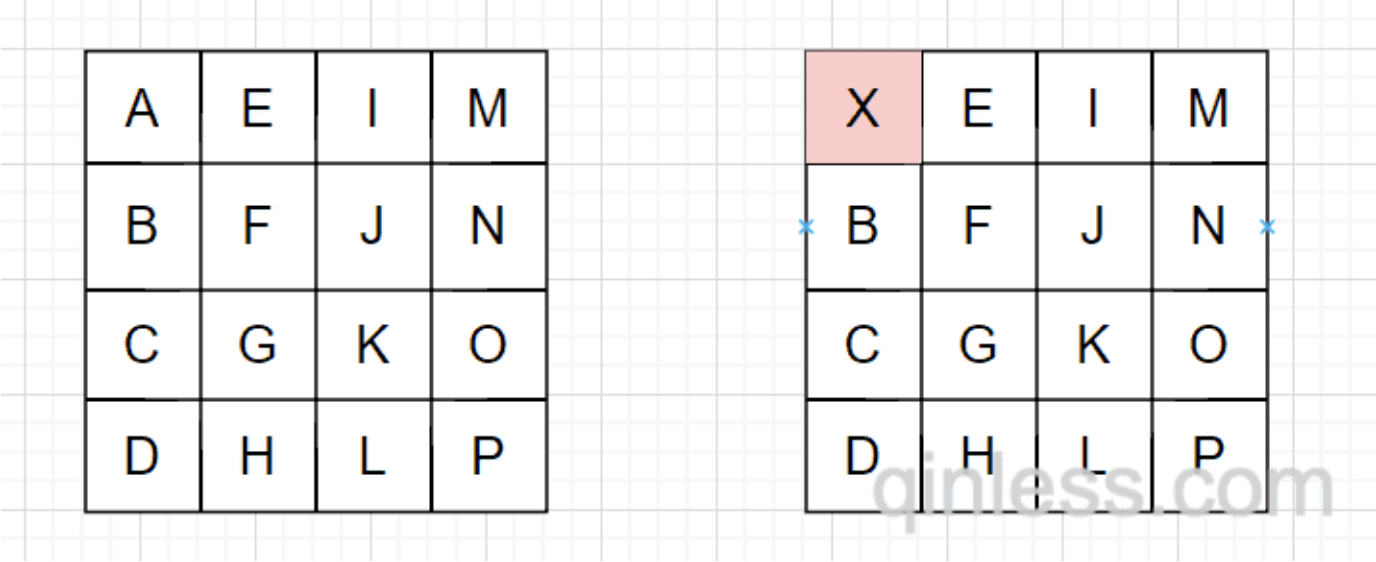
前文已经说过，在AES中，数据以 `state` 的形式计算、中间存储和传输。考虑一个问题，如果在**某个时机**， `state` 中的一个字节发生了错误，变成了另一个字节，这会对密文造成什么影响？

在灰盒攻击模型中，更改 `state` 中一个字节，需要通过设备故障来实现。比如骤变的电压、一个不合时宜的激光束、离子穿透，射线等等。

在白盒攻击模型中，我们可以通过DBI工具（比如Frida），Debuggger（比如IDA）修改二进制文件本身（SO patch）来实现对 `state` 中一个字节的更改，这可以称为引导、诱发一个错误。

返回顶部

因此差分故障攻击或差分错误攻击都是DFA合适的名字，如图是修改 `state` 中第一个字节的值。

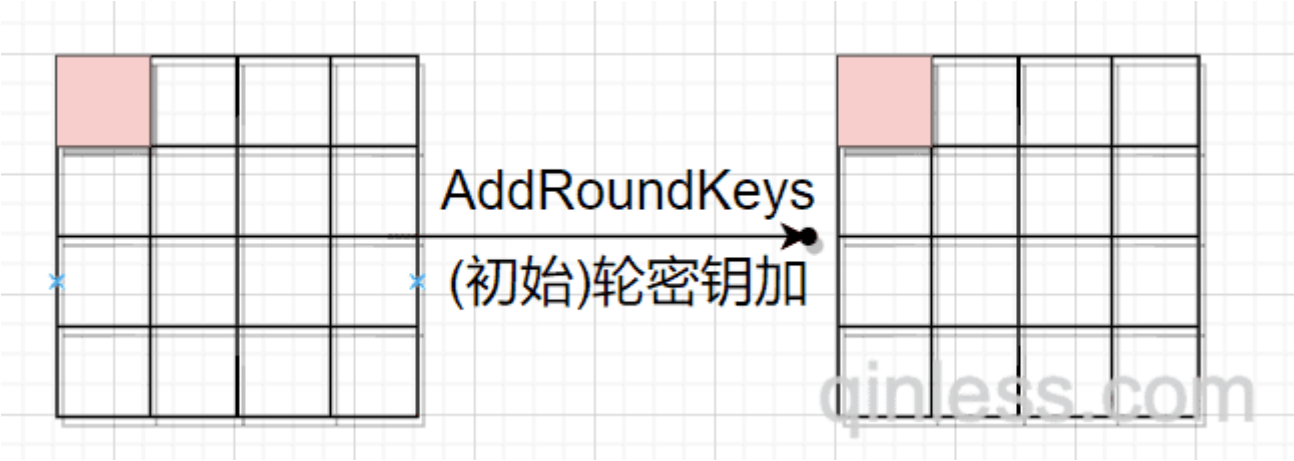


(https://www.qinless.com/wp-content/uploads/2022/04/6247000794b87.png)

先解释一下本节的图示，我们将未修改 `state` 的执行流叫正常情况，修改了 `state` 中一字节的执行流叫故障情况，我们观察正常和故障两种情况下， `state` 中产生差异的字节，即图中的粉色格子。

4.1 起始处发生故障对密文的影响

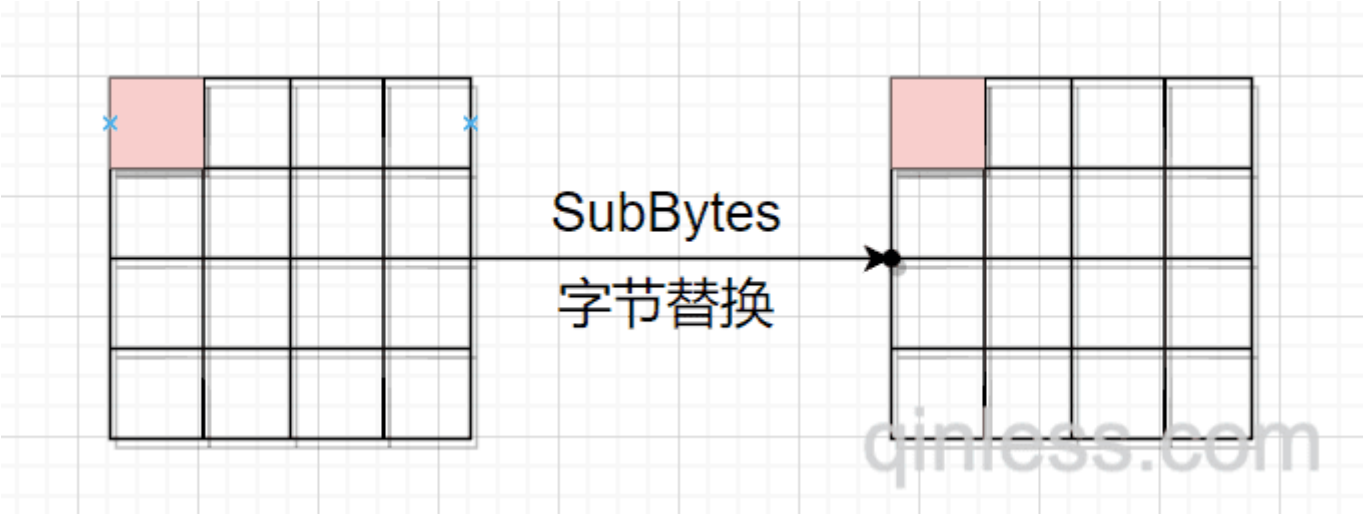
首先是初始轮密钥加，错误限于这一个字节



(https://www.qinless.com/wp-content/uploads/2022/04/6247000cda098.png)

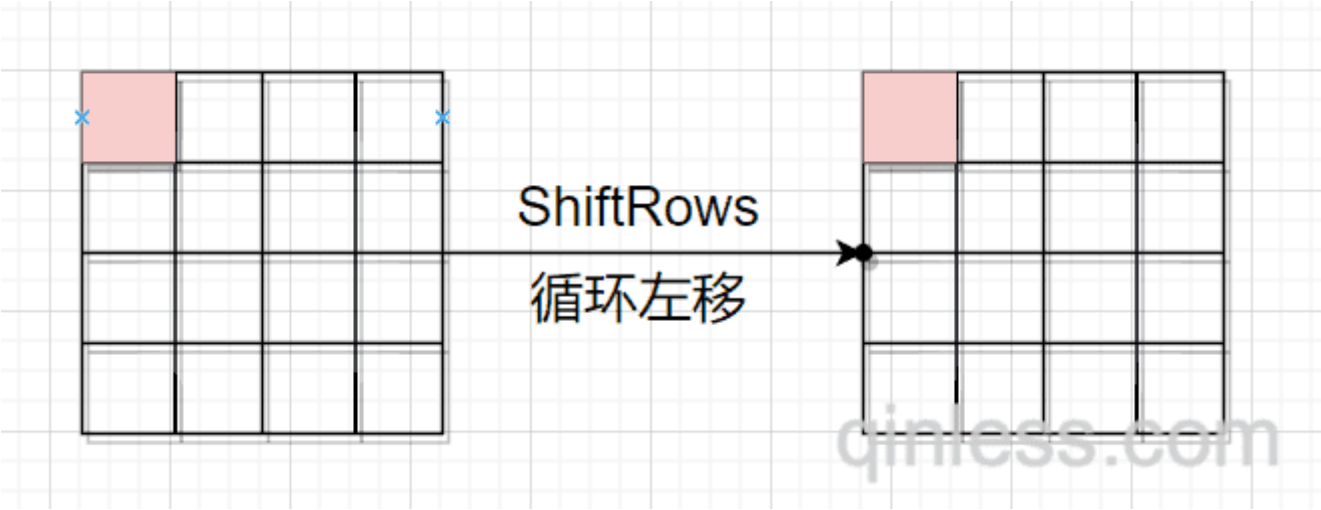
然后是第一轮的字节替换，错误限于这一个字节

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/62470011da60d.png)

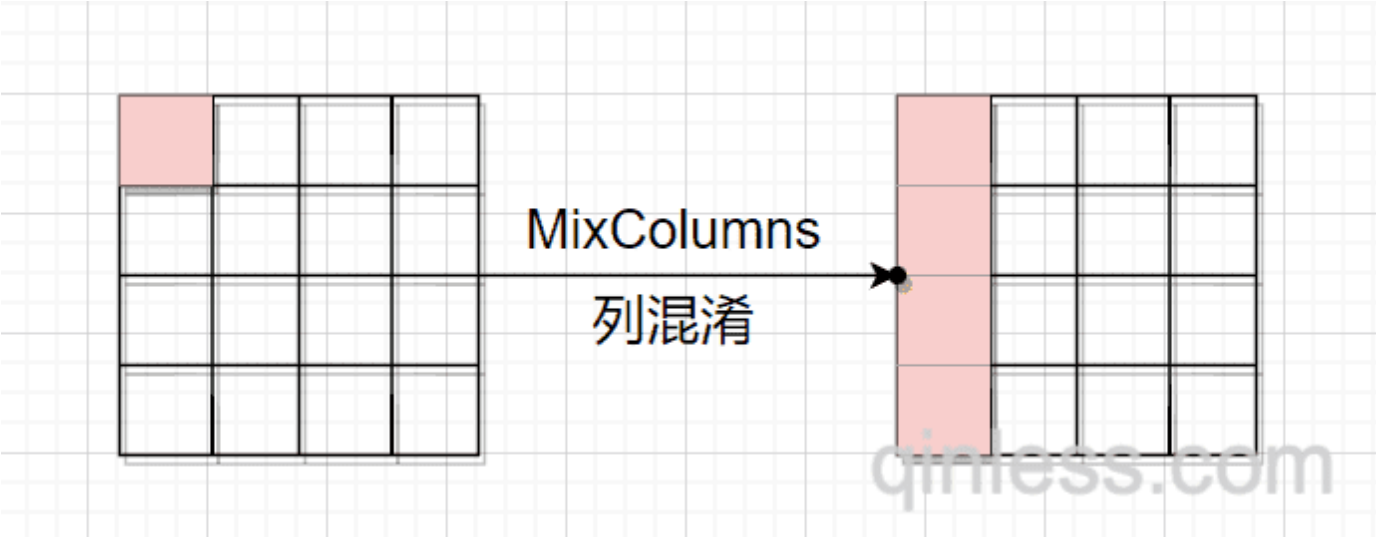
然后是第一轮的循环左移，因为是第一行，所以没动。



(https://www.qinless.com/wp-content/uploads/2022/04/62470016c0dcd.png)

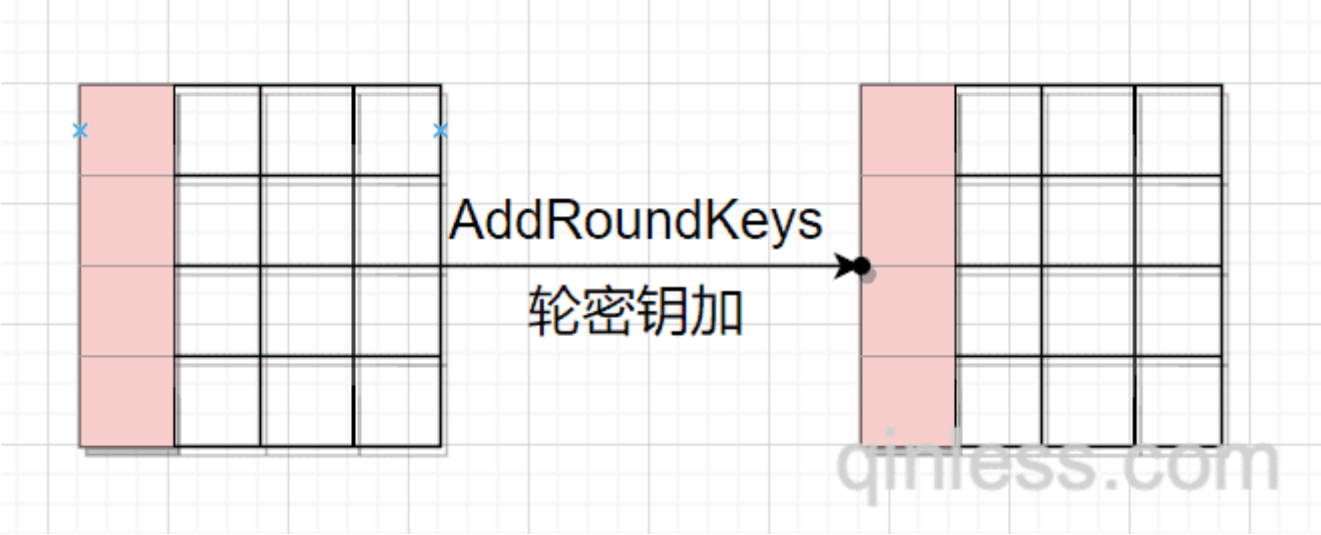
然后是第一轮的列混淆步骤，再次复习矩阵乘法，结果的第m行第n列的值等于**矩阵**A的第m行的元素与**矩阵**B的第n列对应元素乘积之和，因此结果中第一列的每一个元素都受到矩阵B（即下图左边）第一列中每个元素的影响。因而，一个字节的错误被扩散到了一整列。或者说，正常情况和故障情况在第一轮列混淆结束后，有四个字节的值不同。

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6247001be89ce.png)

然后是第一轮的轮密钥加，它只作用用当前字节，不会将差异扩散出去。

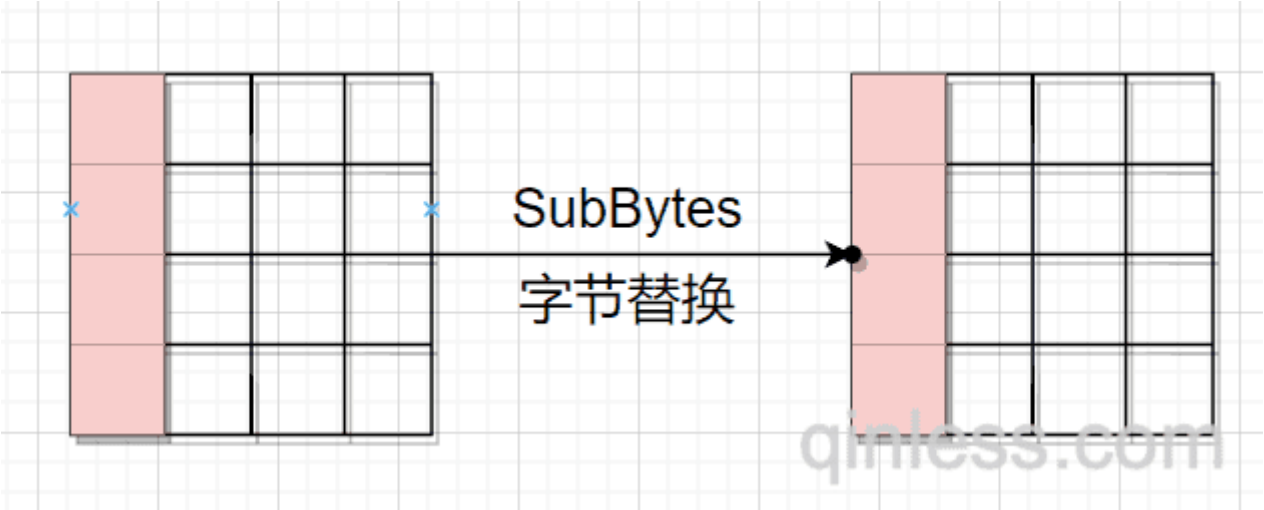


(https://www.qinless.com/wp-content/uploads/2022/04/6247002381b41.png)

可以看到，在一轮循环后，一个字节的故障，被扩散到了四个字节上。我们继续第二轮。

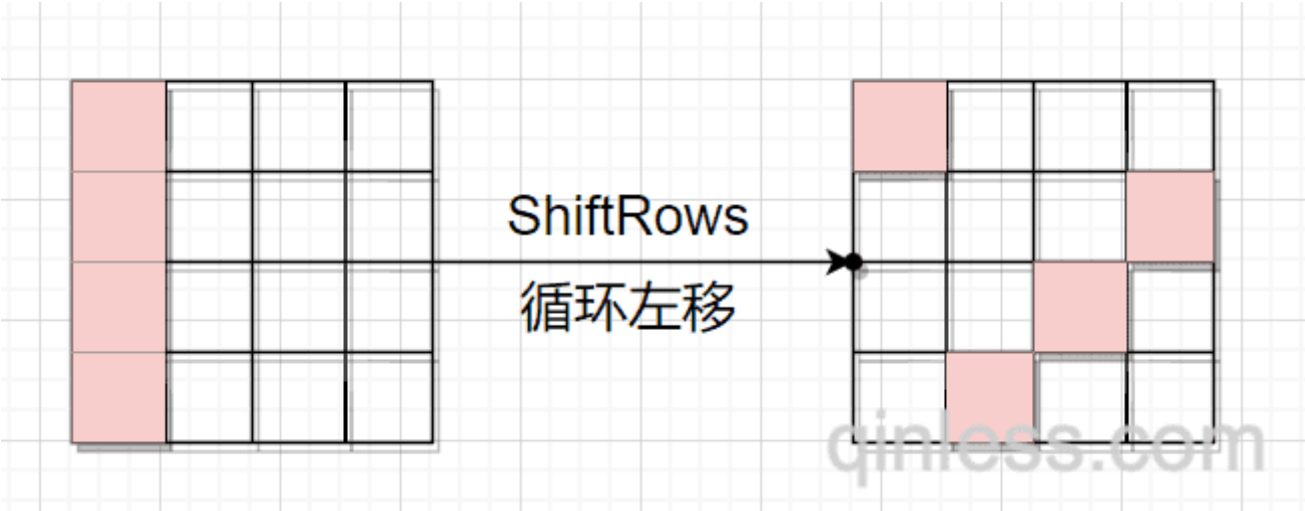
第二轮的字节替换

返回顶部



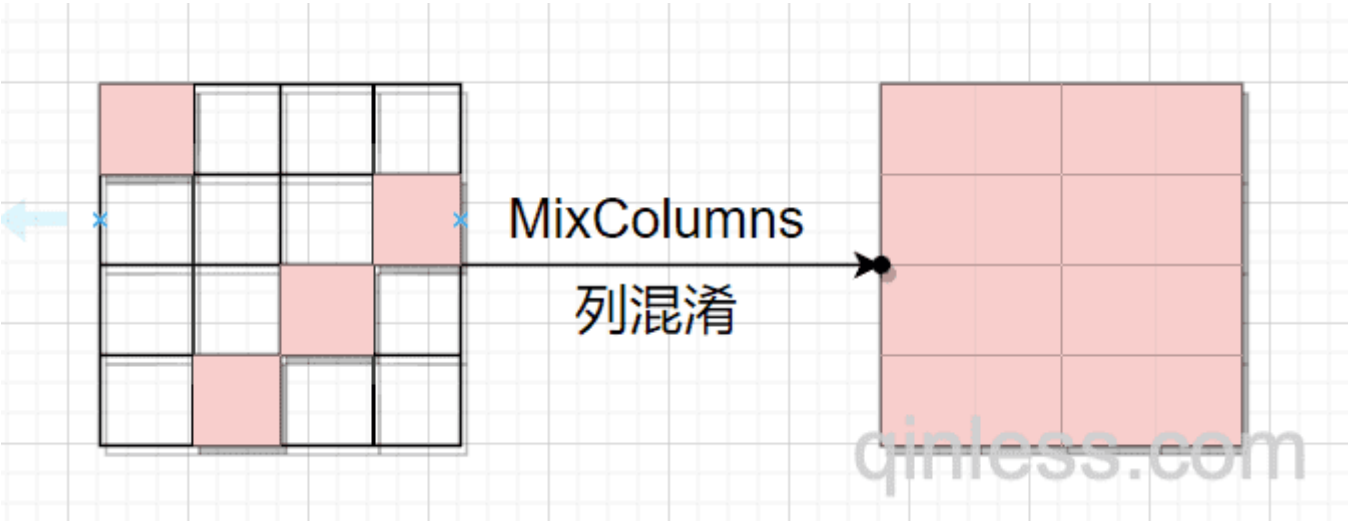
(https://www.qinless.com/wp-content/uploads/2022/04/62470028636b2.png)

第二轮的循环左移，需要注意到，虽然差异还是四个字节，但被扩散到不同的四列去了。



(https://www.qinless.com/wp-content/uploads/2022/04/6247002d433e8.png)

第二轮的列混淆，每列存在的差异扩散到整列，这导致state的全部字节都与原先有差异。



(https://www.qinless.com/wp-content/uploads/2022/04/62470032957d2.png) 返回顶部

后续的就不必说了。

即我们发现，AES运算中，仅在第二轮 $MixColumns$ 结束后，一个字节的差异就已经扩散到所有字节上了。

我们可以更仔细的观察到，如果没有 $MixColumns$ ，那么 $state$ 中一个字节的差异，不论循环多少轮，也只会带来一个字节的差异。反过来说，每一次 $MixColumns$ ，都会让一个字节的差异变成四个字节的差异。

如果没有 $ShiftRows$ ，一列中的变化不会影响其余列。那么state中一个字节的差异，只会带来一列中四个字节的差异。

列混淆和循环左移这两个步骤，提供了 **扩散** 的效果。

4.2 倒数两个列混淆之间发生故障对密文的影响

如果，故障并非发生在开头，而且发生在加密的其他位置呢?

我们关注于倒数两个 $MixColumns$ 中间，即下图这四个时机点中任意一个。

返回顶部

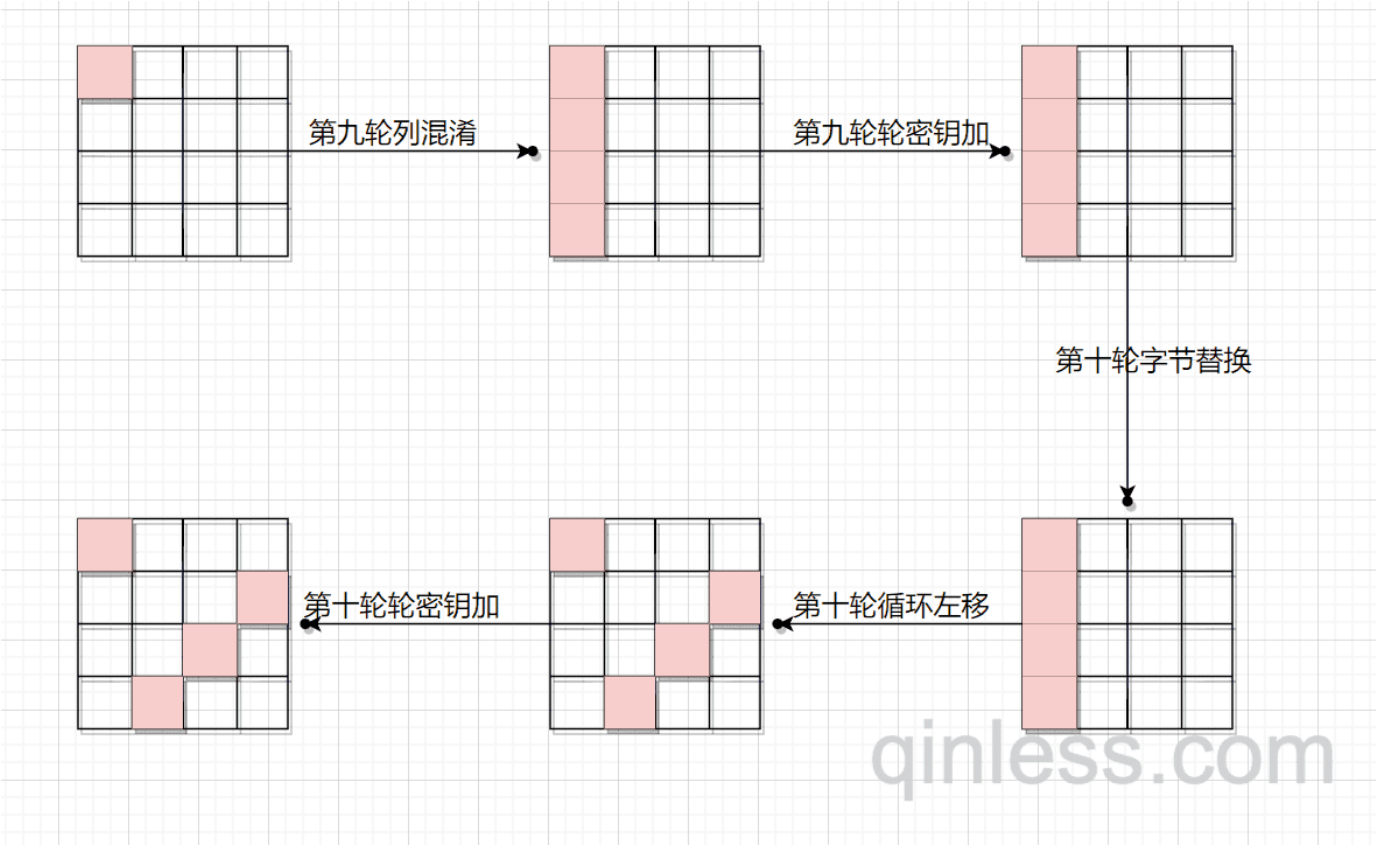
- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey K_8 之前
- SubBytes 之前
- ShiftRows 之前
- MixColumns 之前
- AddRoundKey K_9
- SubBytes
- ShiftRows
- AddRoundKey K_{10}

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/62470038b7221.png)

故障的效果是随机修改state中某一个字节，那么我们应当意识到，图中四个时机点，效果是等价的，这在4.1里就验证过了，只要没经过列混淆，那么错误就只影响一个字节。那么“倒数两个列混淆之间发生故障对密文的影响”只需要看最后一个列混淆前发生故障对密文的影响即可。

返回顶部



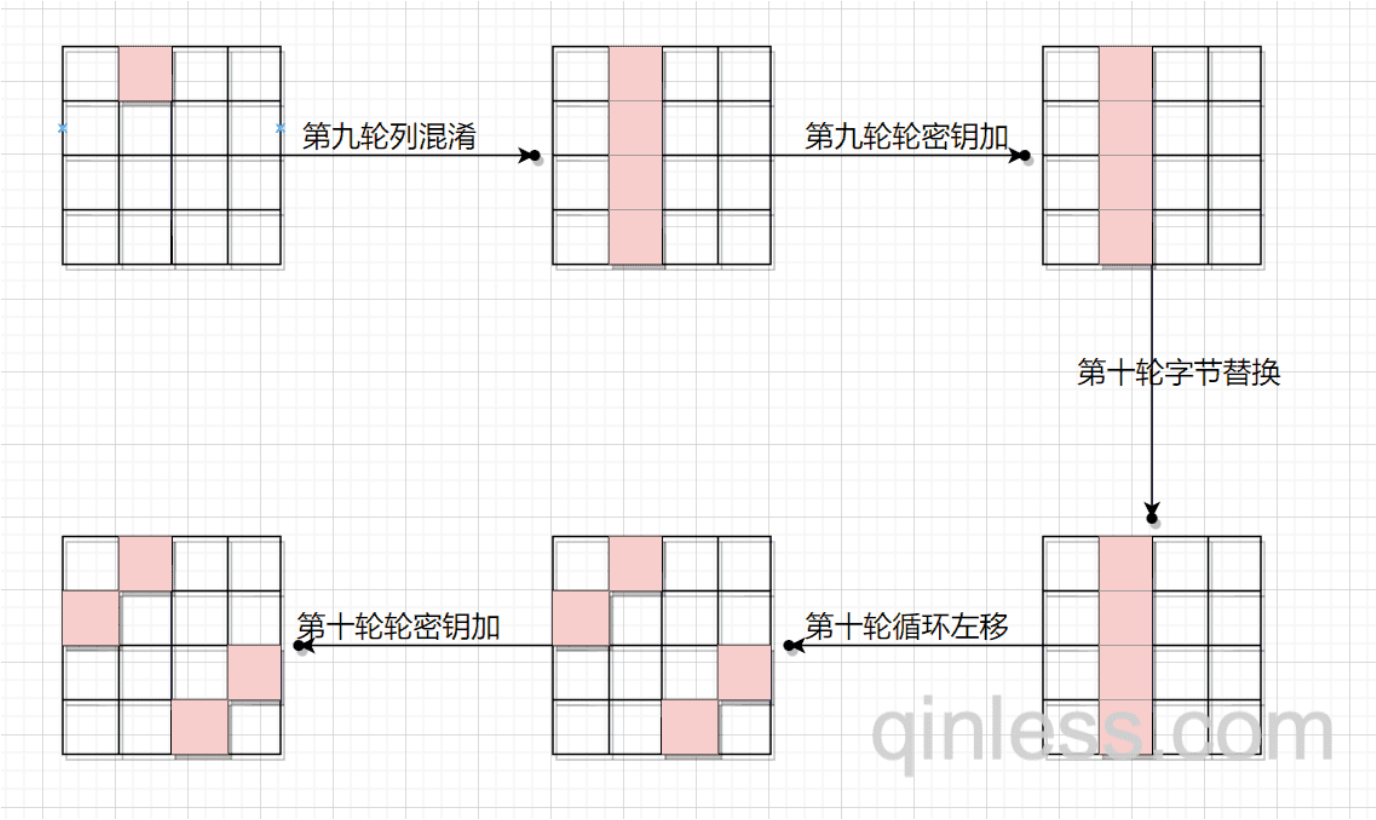
(https://www.qinless.com/wp-content/uploads/2022/04/6247003d8f181.png)

因为后面这五步骤里，只有一次列混淆，所以会导致结果中四个字节改变。这是很好推断的，我们再仔细瞧瞧，有没有别的规律。

如果故障字节位于第一列，那么其最终一定会改变第一、第八、第十一、第十四这四个字节。

如果故障字节在第二列呢

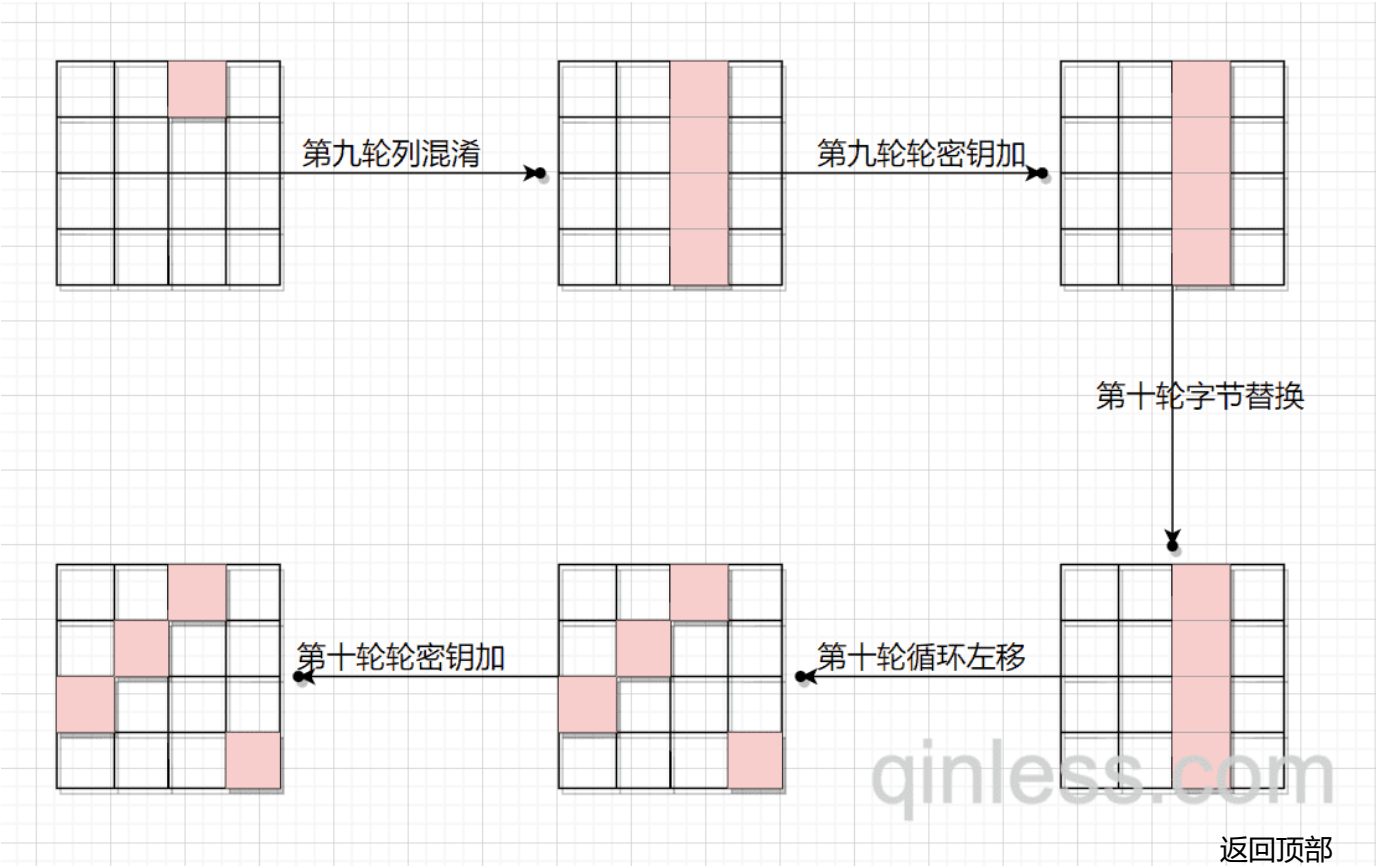
返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/62470042753ef.png)

最终会改变第二、第五、第十二、第十五这四个字节。

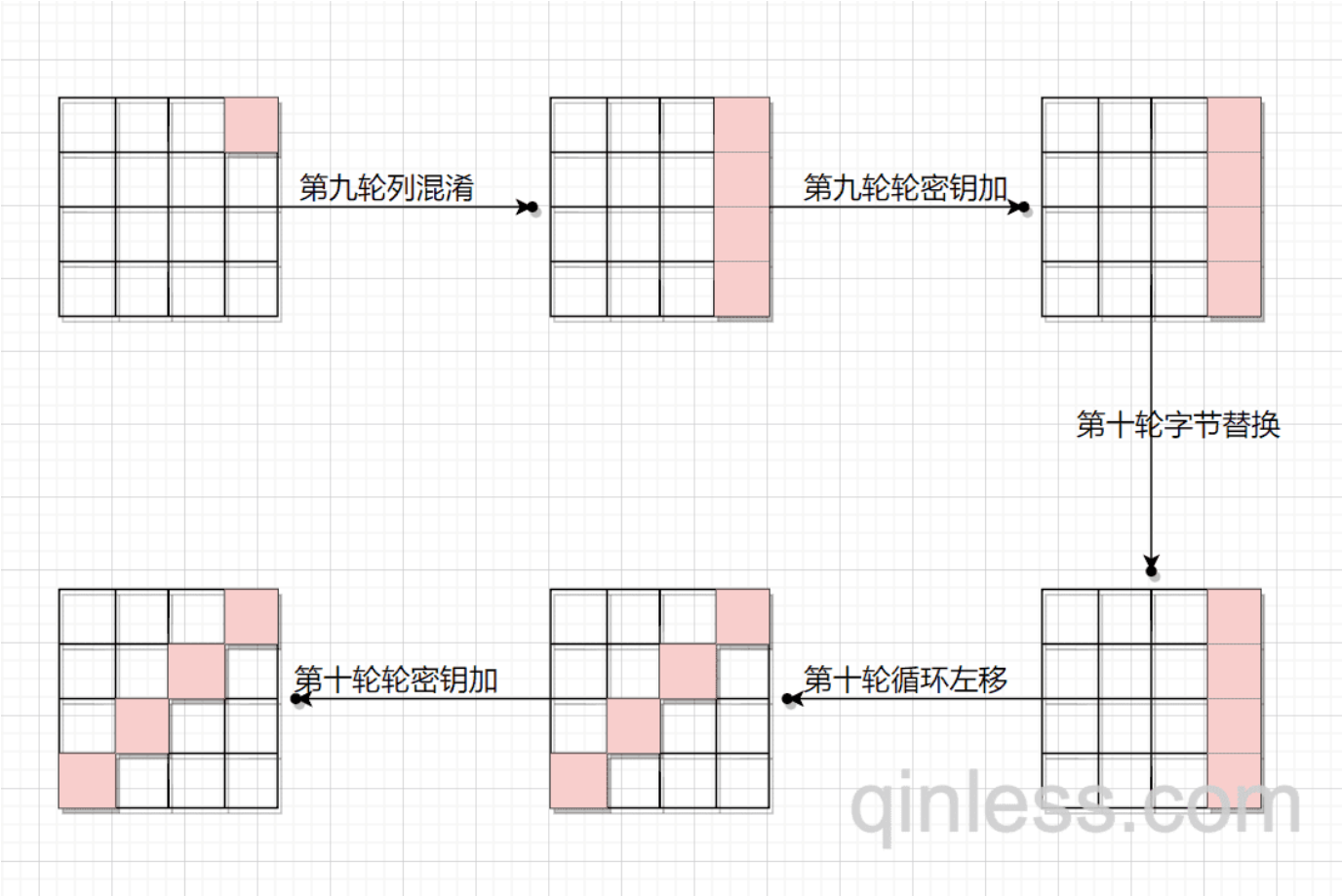
第三列



(https://www.qinless.com/wp-content/uploads/2022/04/62470048404a8.png)

最终会改变第三、第六、第九、第十六这四个字节

第四列



(https://www.qinless.com/wp-content/uploads/2022/04/6247004dbe61d.png)

最终改变第四、第七、第十、第十三这四个字节

我们发现，结果中发生改变的四个字节有其规律，只有四种变法。

这时候我们产生了一种明悟

- 如果故障早于倒数第二个列混淆，那么会影响结果中的十六个字节
- 如果故障发生在倒数两个列混淆之间，那么会影响结果中的四个字节，且依照故障所发生的列，仅存在四种“模式”。
- 如果故障晚于最后一个列混淆，那么会影响结果中的一个字节

4.3 由密文状况反推故障发生地点

基于上面的明悟，我们意识到，可以根据密文的状况，反推故障发生的地点。差分故障攻击中，要求故障发生在4.2中的时机，在实践中如何实现。如果是白盒场景下，我们可以分析二进制代码，在合适的位置Hook实现。那灰盒攻击模型下呢？

返回顶部

bi如智能卡或游戏机，上面存在AES加密，怎么诱发故障，并使得故障总发生在“倒数两个列混淆”之间？难道就随机轰一发电磁炮，让密文和正确密文做对比，是否只影响了四个字节，而且是否符合四种模式之一？

(https://www.qinless.com/wp-content/uploads/2022/04/62470063e7b6e.png)

似乎也不是不行，但感觉成功率不会高。事实上，在实践中有多种办法来提高准确率。

- 并非一定得修改 state 一个字节才能实现“ state 被修改了一字节”的效果，比如故障发生在轮密钥的一个字节上，又或者某一行汇编没执行，即有多种成因会误打误撞实现恰好修改 state 中一个字节的效果。
- 从执行耗时来分析，假设AES加密耗时10ms，“AES的倒数两个列混淆之间”这个时机，应该在8-9ms发生。
- DPA 攻击可以帮助精准定位合适的故障注入时机，这是后面的内容，之后再说。

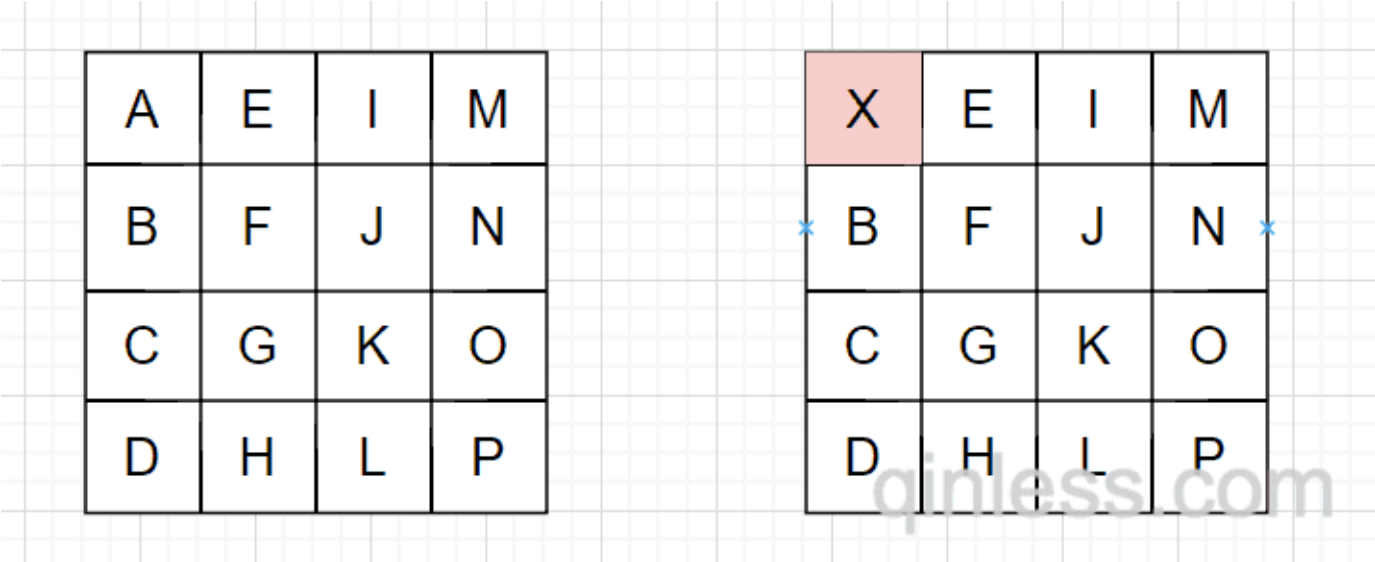
因此我们要说，在灰盒攻击模型中，DFA是可操作的，在白盒模型下更不必说，轻轻松松。第五节中，我们会在第二节写的Python代码上做分析，揭开这些铺垫的目的。

五、差分

在上一节中，我们通过图和方块来表示值的变化，是一种定性的分析，这一节中，我们要从数理的角度定量分析，不要担心，这不会很难。

返回顶部

假设最后一次MixColumns前state状态如下，我们修改第一个字节，A变X



(https://www.qinless.com/wp-content/uploads/2022/04/6247006b07321.png)

首先是列混淆步骤，请复习第二节中所讲的知识。

$$\begin{Bmatrix} 2A+3B+C+D & \dots & \dots & \dots \\ A+2B+3C+D & \dots & \dots & \dots \\ A+B+2C+3D & \dots & \dots & \dots \\ 3A+B+C+2D & \dots & \dots & \dots \end{Bmatrix} and \begin{Bmatrix} 2X+3B+C+D \\ X+2B+3C+D \\ X+B+2C+3D \\ 3X+B+C+2D \end{Bmatrix}$$

接下来是轮密钥加步骤，有些读者可能困惑过，第九轮轮密钥使用 K_9 。

$$\begin{Bmatrix} 2A+3B+C+D+K_{9,0} & \dots & \dots & \dots \\ A+2B+3C+D+K_{9,1} & \dots & \dots & \dots \\ A+B+2C+3D+K_{9,2} & \dots & \dots & \dots \\ 3A+B+C+2D+K_{9,3} & \dots & \dots & \dots \end{Bmatrix} and \begin{Bmatrix} 2X+3B+C \\ X+2B+3C \\ X+B+2C \\ 3X+B+C \end{Bmatrix}$$

接下来是字节替换，这个步骤我们用S函数表示。

$$\begin{Bmatrix} S(2A+3B+C+D+K_{9,0}) & \dots & \dots & \dots \\ S(A+2B+3C+D+K_{9,1}) & \dots & \dots & \dots \\ S(A+B+2C+3D+K_{9,2}) & \dots & \dots & \dots \\ S(3A+B+C+2D+K_{9,3}) & \dots & \dots & \dots \end{Bmatrix} and \begin{Bmatrix} S(2X+3 \\ S(X+2B \\ S(X+B \\ S(3X+B \end{Bmatrix}$$

返回顶部

然后是循环左移

正常 $state$

$$\begin{pmatrix} S(2A+3B+C+D+K_{9,0}) & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & S(A+2B+3C+D+K_{9,1}) \\ \cdots & \cdots & S(A+B+2C+3D+K_{9,2}) & \cdots \\ \cdots & S(3A+B+C+2D+K_{9,3}) & \cdots & \cdots \end{pmatrix}$$

(https://www.qinless.com/wp-content/uploads/2022/04/62470097e1718.png)

故障 $state$

$$\begin{pmatrix} S(2X+3B+C+D+K_{9,0}) & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & S(X+2B+3C+D+K_{9,1}) \\ \cdots & \cdots & S(X+B+2C+3D+K_{9,2}) & \cdots \\ \cdots & S(3X+B+C+2D+K_{9,3}) & \cdots & \cdots \end{pmatrix}$$

(https://www.qinless.com/wp-content/uploads/2022/04/6247009d1b428.png)

最后是末轮的轮密钥加， K_{10}

正常 $state$

$$\begin{pmatrix} (2A+3B+C+D+K_{9,0})+K_{10,0} & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & S(A+2B+3C+D+K_{9,1})+K_{10,13} \\ \cdots & \cdots & S(A+B+2C+3D+K_{9,2})+K_{10,10} & \cdots \\ \cdots & S(3A+B+C+2D+K_{9,3})+K_{10,7} & \cdots & \cdots \end{pmatrix}$$

(https://www.qinless.com/wp-content/uploads/2022/04/624700a274561.png)

故障 $state$

$$\begin{pmatrix} S(2X+3B+C+D+K_{9,0})+K_{10,0} & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & S(X+2B+3C+D+K_{9,1})+K_{10,13} \\ \cdots & \cdots & S(X+B+2C+3D+K_{9,2})+K_{10,10} & \cdots \\ \cdots & S(3X+B+C+2D+K_{9,3})+K_{10,7} & \cdots & \cdots \end{pmatrix}$$

(https://www.qinless.com/wp-content/uploads/2022/04/624700a716c05.png)

我们将正常结果称为 C ，故障结果称为 C' ，十六个字节用下标表示。

$$C_0C_1C_2C_3C_4C_5C_6C_7C_8C_9C_10C_{11}C_{12}C_{13}C_{14}C_{15}$$
$$C'_0C'_1C'_2C'_3C'_4C'_5C'_6C'_7C'_8C'_9C_10'C'_{11}C'_{12}C'_{13}C'_{14}C_{15}'$$

接下来讨论正常和故障密文的差异。

返回顶部

qinless.com

$$
\begin{aligned}
C_0 &= S(2A+3B+C+D+K_{9,0})+K_{10,0}\\
C_0' &= S(2X+3B+C+D+K_{9,0})+K_{10,0}\\
C_7 &= S(3A+B+C+2D+K_{9,3})+K_{10,7}\\
C_7' &= S(3X+B+C+2D+K_{9,3})+K_{10,7}\\
C_{10} &= S(A+B+2C+3D+K_{9,2})+K_{10,10}\\
C_{10}' &= S(X+B+2C+3D+K_{9,2})+K_{10,10}\\
C_{13} &= S(A+2B+3C+D+K_{9,1})+K_{10,13}\\
C_{13}' &= S(X+2B+3C+D+K_{9,1})+K_{10,13}
\end{aligned}
$$

我们先看第一个字节

$$
\begin{aligned}
C_0 &= S(2A+3B+C+D+K_{9,0})+K_{10,0} \quad (1)\\
C_0' &= S(2X+3B+C+D+K_{9,0})+K_{10,0} \quad (2)
\end{aligned}
$$

左右部分上下相加（再次强调，这里的“加”是异或运算）

$$
C_0+C_0' = (S(2A+3B+C+D+K_{9,0})+K_{10,0})+(S(2X+3
K_{10,0})
$$

右式展开

$$
C_0+C_0' = S(2A+3B+C+D+K_{9,0})+S(2X+3B+C+I
K_{10,0} \quad ①
$$

化简

返回顶部

$$
\begin{aligned}
C_0 + C_0' &= S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{10,0}) \\
&= S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{9,0}) + 0 \\
&= S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{9,0})
\end{aligned}
$$

两次异或同样的数，等于异或0，再等于没做异或。下面反其道而行之，老实说，对于数学渣而言，每次看到这类操作，都感到

(https://www.qinless.com/wp-content/uploads/2022/04/6247007f15a33.png)

$$
\begin{aligned}
&S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{9,0}) \\
&= S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{9,0} + 0) \\
&= S(2A + 3B + C + D + K_{9,0}) + S(2X + 3B + C + D + K_{9,0} + 2A)) \\
&= S(2A + 3B + C + D + K_{9,0}) + S(2A + 3B + C + D + K_{9,0} + 2X)
\end{aligned}
$$

令 $Y_0 = 2A + 3B + C + D + K_{9,0}$ ， $Z = A + X$ 。

$$
C_0 + C_0' = S(Y_0) + S(Y_0 + 2Z) \quad (3)
$$

返回顶部

这里我们要引入差分的概念， A 是正确的输入值， X 是故障带来的输入值， A 与 X 异或的结果我们叫它输入差分。同理， C_o 是正确的密文， C_0' 是故障带来的密文， C_0 与 C_0' 异或的结果叫它输出差分。

读者不必对这些概念感到恐慌，我们只是为了更好的指代 $C_0 + C_0'$ 以及 $A + X$ ，它们在后面要被多次使用。

考虑(3)式中的已知量与未知量，在灰盒攻击模型中，我们能观测加密的结果，但观察不到state的中间量或故障导致的变化。因此正确密文和故障密文都可知，正确输入和故障输入都未知。进而，输入差分未知，输出差分已知。式子左边已知，而右边的 Y_0 以及输出差分 Z 未知。

关键点来了， Y_0 和 Z 都是单个字节，即0-255之间的数。可是，是否任意的 Z 都可以带来左边的结果?

假设差分是0x10，Python代码验证如下

返回顶部

COPY

```
SBox = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
]

# 传入两个数，计算S(x) ^ S(y)
def getSDiff(x, y):
    return SBox[x] ^ SBox[y]

# 乘2，前面讲过的乘法规则
def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b

    return res % 0x100

diff = 0x10
# 使用集合去重
zlist = set({})
# 遍历z
for z in range(0x100):
    # 遍历y0
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == diff:
            zlist.add(z)
```

返回顶部

```
print(len(zlist))

# run result:127
```

可以发现，输出差分为0x10的约束下，Z的取值空间并非0-255Z范围内全部数字，仅有127个值符合条件。

有没有办法进一步约束 Z 的范围？别忘了我们才用了一个约束，还有三个输出差分可以用。

$$
\begin{aligned}
C_0 &= S(2A+3B+C+D+K_{9,0})+K_{10,0} \\
C_0' &= S(2X+3B+C+D+K_{9,0})+K_{10,0} \\
C_7 &= S(3A+B+C+2D+K_{9,3})+K_{10,7} \\
C_7' &= S(3X+B+C+2D+K_{9,3})+K_{10,7} \\
C_{10} &= S(A+B+2C+3D+K_{9,2})+K_{10,10} \\
C_{10}' &= S(X+B+2C+3D+K_{9,2})+K_{10,10} \\
C_{13} &= S(A+2B+3C+D+K_{9,1})+K_{10,13} \\
C_{13}' &= S(X+2B+3C+D+K_{9,1})+K_{10,13}
\end{aligned}
$$

对后面三条做同样的操作

$$
\begin{aligned}
\text{令}Y1 &= 3A+B+C+2D+K_{9,3} \\
C_7+C_7' &= S(Y_1)+S(Y_1+3Z) \\
\text{令}Y_2 &= A+B+2C+3D+K_{9,2} \\
C_{10}+C_{10}' &= S(Y_2)+S(Y_2+Z) \\
\text{令}Y_3 &= A+2B+3C+D+K_{9,1} \\
C_{13}+C_{13}' &= S(Y_3)+S(Y_3+Z)
\end{aligned}
$$

C_7+C_7' ， $C_{10}+C_{10}'$ ， $C_{13}+C_{13}'$ 都是已知的，那么和 C_0+C_0' 类似，也可以对输入差分 Z 起到约束作用。

Y_1 ， Y_2 ， Y_3 都在0-255之间，如法炮制。

返回顶部

接下来用第二节中的AES Python代码，对第一个字节做故障注入

正常密文

```
8d f4 e9 aa c5 c7 57 3a 27 d8 d0 55 d6 e4 d6 4b
```

```
def encrypt(input_bytes, kList):
    '''


    :param input_bytes: 输入的明文
    :param kList: K0-K10
    :return:
    '''
    plainState = text2matrix(input_bytes)
    # 初始轮密钥加
    state = AddRoundKeys(plainState, kList[0:4])
    for i in range(1, 10):
        state = SubBytes(state)
        state = ShiftRows(state)
        # 第九轮列混淆前将state第一个字节改为0
        if(i==9):
            state[0][0] = 0
        state = MixColumns(state)
        state = AddRoundKeys(state, kList[4 * i:4 * (i + 1)])

    state = SubBytes(state)
    state = ShiftRows(state)
    state = AddRoundKeys(state, kList[40:44])
    return state
```

注入故障后的密文

```
3c f4 e9 aa c5 c7 57 a5 27 d8 2e 55 d6 36 d6 4b
```

结果的第一个字节是0x8d，变成了0x3c

结果的第八个字节是0x3a，变成了0xa5

结果的第十一个字节是0xd0，变成了0x2e

结果的第十四个字节是0xe4，变成了0x36

返回顶部

COPY

```
SBox = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
]

# 传入两个数，计算S(x) ^ S(y)
def getSDiff(x, y):
    return SBox[x] ^ SBox[y]

def mul_by_01(num):
    return num

def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b

    return res % 0x100

def mul_by_03(num):
    return (mul_by_02(num) ^ num)

z1list = set({})
z2list = set({})
z3list = set({})
z4list = set({})
zlist = set({})
for z in range(0x100):
```

返回顶部

```
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0x3c):
            z1list.add(z)

    for y1 in range(0x100):
        tmp = getSDiff(y1, mul_by_03(z) ^ y1)
        if tmp == (0x3a ^ 0xa5):
            z2list.add(z)

    for y3 in range(0x100):
        tmp = getSDiff(y3, z ^ y3)
        if tmp == (0xd0 ^ 0x2e):
            z3list.add(z)

    for y4 in range(0x100):
        tmp = getSDiff(y4, z ^ y4)
        if tmp == (0xe4 ^ 0x36):
            z4list.add(z)

print(len(z1list))
print(len(z2list))
print(len(z3list))
print(len(z4list))

# run result
# 127
# 127
# 127
# 127
```

z1list 到 z4list是**分别**满足1式到4式的 Z 的集合，可以发现分别有127个元素满足要求。需要意识到，尽管输入差分未知，但都是 A 变 X 这个故障带来的，即每个式子中的输入差分Z是同一个Z，Z必须同时满足这四个式子，即 Z 的范围是这四个集合的交集。Python提供了非常方便的API。

返回顶部

```
zlist = set.intersection(z1list, z2list, z3list, z4list)
print(len(zlist))
print(zlist)
# result
# 15
# {193, 132, 68, 134, 7, 197, 105, 205, 109, 145, 82, 55, 59, 156, 61}
```

我们惊喜的发现， Z 的范围被缩小到了15个值，或者说十五个候选 Z 。

大家可能会有两个困惑

- 为什么要去缩小 Z 的范围，甚至是求 Z 具体的值?
- 如何进一步缩小 Z 的值。

先看第一个问题，如下式子大家应该不陌生，它是用来约束 Z 取值范围的四个式子之一。

$$C_0 + C_0' = S(Y_0) + S(Y_0 + 2Z) \quad (3)$$

那么，当 Z 范围缩小，甚至固定时，是不是同理可以约束 Y_0 的取值范围呢？即左边的输出差分已知且固定， Z 有取值限制，那么0-255范围内的 Y_0 是不是都能构建等式呢？我想可以验证一下

Z_0 现在只有15个可选项，遍历寻找符合条件的 Y_0

```
y0list = set({})
for z in zlist:
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0x3c):
            y0list.add(y0)

print(len(y0list))
print(y0list)
# 30
# {128, 1, 3, 131, 8, 137, 140, 141, 16, 145, 154, 178, 181, 186, 60, 64, 70, 71, 78, 215, 220,
```

返回顶部

我们惊喜的发现， Y_0 的取值范围被约束到30个潜在值里了。别忘了还有下面这个式子

$$C_0 = S(2A + 3B + C + D + K_{9,0}) + K_{10,0}$$

即

$$C_0 = S(Y_0) + K_{10,0}$$

那么我们可以将 $K_{10,0}$ 也限制在30个值之中。在上面密钥的相关问题讨论中，我们已经验证，AES-128可以依靠一轮的轮密钥逆推出主密钥。因此问题就变成了——如果能尽可能约束 Z 的取值范围，或许能让 $K_{10,0}$ 唯一，同理对其余15个K做同样的事，恢复轮密钥，最终逆推主密钥。

大家应该理解为什么要约束 Z 了，再看第二个问题——如何进一步约束 Z

首先我们发现，对 Z 具有约束条件的四个式子都被我们“用完”了。能不能另外再来一些式子呢？比如重新再注入一个故障，不是又有四组等式了吗？这是行不通的，不要忘了，重新注入故障的话，输入差分 Z 已经变了，新的等式不能再对原来的 Z 取值范围产生约束。

那么不妨换个思路， Z 是变化的，无法通过多次重新注入来缩小范围，但是 $K_{10,n}$ 不会变化！因为密钥始终是那个密钥。所以可以通过多次注入来缩小 $K_{10,n}$ 的范围。

在刚才的故障注入中，我们将 $K_{10,0}$ 的候选人缩小到30个值之中。那么考虑一下，如果再注入故障，新的 $K_{10,0}$ 可以如法炮制出一个取值范围，真正的 $K_{10,0}$ 是哪个？我们不知道。但它必定在第一次那30个候选人里，也一定在第二个那30个候选人里面。换言之，它在两个范围的交集里。

首先由 Y_0 计算出对应的 $K_{10,0}$

$$\begin{gathered} C_0 = S(Y_0) + K_{10,0} \\ S(Y_0) + C_0 = S(Y_0) + S(Y_0) + K_{10,0} \\ S(Y_0) + C_0 = K_{10,0} \end{gathered}$$

返回顶部

```
k = set({})
for y0 in list(y0list):
    k.add(SBox[y0] ^ 0x8d)

print(k)
# {131, 132, 11, 12, 19, 20, 155, 156, 162, 165, 42, 45, 50, 53, 186, 189, 64, 71, 200, 207, 20
```

完整代码如下

返回顶部

COPY

```
SBox = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
]

# 传入两个数，计算S(x) ^ S(y)
def getSDiff(x, y):
    return SBox[x] ^ SBox[y]

def mul_by_01(num):
    return num

def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b

    return res % 0x100

def mul_by_03(num):
    return (mul_by_02(num) ^ num)

z1list = set({})
z2list = set({})
z3list = set({})
z4list = set({})
zlist = set({})
for z in range(0x100):
```

返回顶部

```
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0x3c):
            z1list.add(z)

    for y1 in range(0x100):
        tmp = getSDiff(y1, mul_by_03(z) ^ y1)
        if tmp == (0x3a ^ 0xa5):
            z2list.add(z)

    for y3 in range(0x100):
        tmp = getSDiff(y3, z ^ y3)
        if tmp == (0xd0 ^ 0x2e):
            z3list.add(z)

    for y4 in range(0x100):
        tmp = getSDiff(y4, z ^ y4)
        if tmp == (0xe4 ^ 0x36):
            z4list.add(z)

zlist = set.intersection(z1list, z2list, z3list, z4list)

y0list = set({})
for z in zlist:
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0x3c):
            y0list.add(y0)

print(len(y0list))
print(y0list)

k = set({})
for y0 in list(y0list):
    k.add(SBox[y0] ^ 0x8d)

print("k range")
print(k)
```

重新注入一个新的故障，同样在 $state$ 的第一个字节

返回顶部

```
def encrypt(input_bytes, kList):
    '''


    :param input_bytes: 输入的明文
    :param kList: K0-K10
    :return:
    '''
    plainState = text2matrix(input_bytes)
    # 初始轮密钥加
    state = AddRoundKeys(plainState, kList[0:4])
    for i in range(1, 10):
        state = SubBytes(state)
        state = ShiftRows(state)
        # 第九轮列混淆前将state第一个字节改为1
        if(i==9):
            state[0][0] = 1
        state = MixColumns(state)
        state = AddRoundKeys(state, kList[4 * i:4 * (i + 1)])

    state = SubBytes(state)
    state = ShiftRows(state)
    state = AddRoundKeys(state, kList[40:44])
    return state
```

正确密文依然 8d f4 e9 aa c5 c7 57 3a 27 d8 d0 55 d6 e4 d6 4b

故障密文变成 dc f4 e9 aa c5 c7 57 0a 27 d8 26 55 d6 ad d6 4b

0x8d 0xdc

0x3a 0x0a

0xd0 0x26

0xe4 0xad

同理计算K的范围

返回顶部

COPY

```
SBox = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0
]

# 传入两个数，计算S(x) ^ S(y)
def getSDiff(x, y):
    return SBox[x] ^ SBox[y]

def mul_by_01(num):
    return num

def mul_by_02(num):
    if num < 0x80:
        res = (num << 1)
    else:
        res = (num << 1) ^ 0x1b

    return res % 0x100

def mul_by_03(num):
    return (mul_by_02(num) ^ num)

z1list = set({})
z2list = set({})
z3list = set({})
z4list = set({})
zlist = set({})
for z in range(0x100):
```

返回顶部

```
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0xdc):
            z1list.add(z)

    for y1 in range(0x100):
        tmp = getSDiff(y1, mul_by_03(z) ^ y1)
        if tmp == (0x3a ^ 0x0a):
            z2list.add(z)

    for y3 in range(0x100):
        tmp = getSDiff(y3, z ^ y3)
        if tmp == (0xd0 ^ 0x26):
            z3list.add(z)

    for y4 in range(0x100):
        tmp = getSDiff(y4, z ^ y4)
        if tmp == (0xe4 ^ 0xad):
            z4list.add(z)

zlist = set.intersection(z1list, z2list, z3list, z4list)

y0list = set({})
for z in zlist:
    for y0 in range(0x100):
        tmp = getSDiff(y0, mul_by_02(z) ^ y0)
        if tmp == (0x8d^0xdc):
            y0list.add(y0)

print(len(y0list))
print(y0list)

k = set({})
for y0 in list(y0list):
    k.add(SBox[y0] ^ 0x8d)

print("k range")
print(k)
# {129, 130, 4, 7, 16, 19, 149, 150, 168, 171, 45, 46, 57, 58, 188, 65, 66, 196, 199, 208, 211,
```

返回顶部

将两个 K 的候选范围取交集

```
klist1 = {131, 132, 11, 12, 19, 20, 155, 156, 162, 165, 42, 45, 50, 53, 186, 189, 64, 71, 200,
klist2 = {129, 130, 4, 7, 16, 19, 149, 150, 168, 171, 45, 46, 57, 58, 188, 65, 66, 196, 199, 20
klist = set.intersection(klist1,klist2)
print(klist)
# result
# {208, 19, 45}
```

我们惊喜的发现， $K_{10,0}$ 的候选项只剩三个了，再注入一次故障。 $state$ 同样位置改成3。

```
def encrypt(input_bytes, kList):
    '''


    :param input_bytes: 输入的明文
    :param kList: K0-K10
    :return:
    '''
    plainState = text2matrix(input_bytes)
    # 初始轮密钥加
    state = AddRoundKeys(plainState, kList[0:4])
    for i in range(1, 10):
        state = SubBytes(state)
        state = ShiftRows(state)
        # 第九轮列混淆前将state第一个字节改为3
        if(i==9):
            state[0][0] = 3
        state = MixColumns(state)
        state = AddRoundKeys(state, kList[4 * i:4 * (i + 1)])

    state = SubBytes(state)
    state = ShiftRows(state)
    state = AddRoundKeys(state, kList[40:44])
    return state
```

第三个故障注入得到的 $K_{10,0}$ 范围如下

```
{18, 147, 148, 21, 26, 155, 156, 29, 162, 35, 165, 170, 43, 44, 173, 208, 81, 86, 215, 216, 89,
```

返回顶部

三次故障注入得到的 $K_{10,0}$ 范围取交集

```
klist1 = {131, 132, 11, 12, 19, 20, 155, 156, 162, 165, 42, 45, 50, 53, 186, 189, 64, 71, 200,
klist2 = {129, 130, 4, 7, 16, 19, 149, 150, 168, 171, 45, 46, 57, 58, 188, 65, 66, 196, 199, 20
klist3 = {18, 147, 148, 21, 26, 155, 156, 29, 162, 35, 165, 170, 43, 44, 173, 208, 81, 86, 215,
klist = set.intersection(klist1,klist2,klist3)
print(klist)
# result
# {208}
```

结果就一个元素，208，即0xd0。这正是我们的demo中，Key在编排后， K_{10} 第一个字节。仅依靠上面的三个注入，我们还可以求出 $K_{10,7}, K_{10,10}, K_{10,13}$ 。因为对 K 约束的式子里，我们上面只关注了 $K_{10,0}$ 。而当故障注入发生在第二列、第三列、第四列时，就可以求出另外12个 K_{10} 中的字节，进而获得完整的 K_{10} 。在最好的情况下，只需要八次故障注入，就可以还原完整的 K_{10} 。不那么完美的情况下，数十次，成百上千次故障注入也是可接受的，成本并不高。

很感谢看到这里的读者朋友们，密码攻击过去对大家可能是很遥远的事，但上文中我们一起完成了差分故障攻击。可以发现，事实上也并没有太难。差分故障攻击是灰盒攻击中的技术，为什么在白盒攻击的系列文章里，要花那么大力气学它呢？自然是因为白盒加密中可以使用它剥离密钥。下一节中我们会进行阐述。

六、工具选择

我们希望传入正确密文和一系列故障密文，就直接返回第十轮的轮密钥甚至是主密钥。上面写的demo部分实现了功能，但代码不够完善。phoenixAES (https://github.com/SideChannelMarvels/JeanGrey/tree/master/phoenixAES) 可以很好的完成需求，它会返回最后一轮的轮密钥，配合上面介绍的 $stark$ 就可以恢复主密钥。

phoenixAES 可以通过pip直接安装，tracefile中，第一行传正确密文，后面传入故障密文。

```
import phoenixAES

with open('tracefile', 'wb') as t:
    t.write("""77432fc75482b8f07009389f2ce7c8f4
90432fc75482b82d70093b9f2c3cc8f4
0b432fc75482b8547009339f2c6bc8f4
    """.encode('utf8'))

phoenixAES.crack_file('tracefile', [], True, False, 3)
```

返回顶部

这个工具有三个优点

1是良好的输出日志，比如上面的情况中，我们只提供了两组故障密文，无法还原出完整的轮密钥，那么它会打印出已还原的部分，未还原的部分用星号表示。

```
Run:  demo
K0: ['0B', '77'] K7: ['80', '24'] Ka: ['AC', 'A7'] Kd: ['53', 'DF']
K0: ['8A', 'F6'] K7: ['1A', 'BE'] Ka: ['B4', 'BF'] Kd: ['F8', '74']
K0: ['D2', 'AE'] K7: ['71', 'D5'] Ka: ['BC', 'B7'] Kd: ['1B', '97']
K0: ['C3', 'BF'] K7: ['E8', '4C'] Ka: ['49', '42'] Kd: ['D0', '5C']
K0: ['72', '0E'] K7: ['FA', '5E'] Ka: ['9D', '96'] Kd: ['A8', '24']
K0: ['5B', '27'] K7: ['43', 'E7'] Ka: ['40', '4B'] Kd: ['8A', '06']
K0: ['FB', '87'] K7: ['C8', '6C'] Ka: ['61', '6A'] Kd: ['39', 'B5']
K0: ['EA', '96'] K7: ['51', 'F5'] Ka: ['94', '9F'] Kd: ['F2', '7E']
K0: ['81', 'FD'] K7: ['C2', '66'] Ka: ['04', '0F'] Kd: ['38', 'B4']
K0: ['19', '65'] K7: ['69', 'CD'] Ka: ['0D', '06'] Kd: ['62', 'EE']
K0: ['B9', 'C5'] K7: ['E2', '46'] Ka: ['2C', '27'] Kd: ['D1', '5D']
K0: ['08', '74'] K7: ['F0', '54'] Ka: ['F8', 'F3'] Kd: ['A9', '25']
K0: ['A8', 'D4'] K7: ['7B', 'DF'] Ka: ['D9', 'D2'] Kd: ['1A', '96']
K0: ['21', '5D'] K7: ['49', 'ED'] Ka: ['25', '2E'] Kd: ['8B', '07']
K0: ['30', '4C'] K7: ['D0', '74'] Ka: ['D0', 'DB'] Kd: ['40', 'CC']
K0: ['90', 'EC'] K7: ['5B', 'FF'] Ka: ['F1', 'FA'] Kd: ['F3', '7F']
K0: ['4A', '36'] K7: ['DA', '7E'] Ka: ['B5', 'BE'] Kd: ['41', 'CD']
Round key bytes recovered:
A0............68....09....E5....
```

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/6247008cb175b.png)

2是会检测输入的故障密文是否合规，可用就用，不可用就放弃此密文。如果错误密文为四字节，且是四种模式之一，那么我们知道这是一个正确时机的故障，工具会输出 good candidate。如果时机晚，错误密文少于四字节，工具会提示 too few impact。时机早，错误密文超过四字节，会提示 too much impact。因此我们可以做自动化，随机化时机注入故障，获得大量故障密文，尽管其中大部分时机不对，没法用。phoenixAES会替我们筛选出可用的，正确时机的故障密文并计算出密钥。

3是它支持加密和解密，本文只讲了加密，解密其实技术上也类似。crack_file函数的第三个参数传False即代表解密。在解密情况中，首行放正确明文，后面是故障明文。

除此之外还有其他的开源工具可以处理DFA，但都不够清爽利索，把简单的事弄繁琐了。

六、尾声

返回顶部

本文对于数学或密码学相关专业人员来说太过拖沓，且深度不够，几段话就能讲清的事用了大量笔墨。但必须要说的是，这正是我初学DFA时最想看到的那篇文章，我想我把它写出来了。

标签: # aes, # dfa, # frida, # unidbg, # 差分故障攻击, # 攻击模型, # 白盒aes (https://www.qinless.com/tag/baiheaes)

(https://www.qinless.com/1647)

(https://www.qinless.com/wp-content/themes/honey/public/qrcode?data=https://www.qinless.com/1647)

(https://www.qinless.com/1647) 暂无评论 (https://www.qinless.com/1647#respond)

【转载】第二讲——白盒加密攻击方法的选择

上一篇 (https://www.qinless.com/1645)

【转载】第四讲——差分故障攻击的工具链

下一篇 (https://www.qinless.com/1653)

会爬山的小脑虎 (https://www.qinless.com/author/xiayu)

小爬爬

本文作者： **会爬山的小脑虎 (https://www.qinless.com/author/xiayu)**
本文链接： **https://www.qinless.com/?p=1647 (https://www.qinless.com/1647)**
版权声明：本博客所有文章除特别声明外，均采用 CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/) 许可协议。转载请注明来自 qinless 的博客！ (https://www.qinless.com)

返回顶部

微信搜一搜

逆向小友

发表评论

昵称 *

邮箱 *

网址

输入评论内容...

提交评论

Copyright 2022. All Rights Reserved. Powered By XinTheme (http://www.xintheme.com) + WordPress 果酱 (https://blog.wpjam.com/) 浙ICP备2021029793号 (http://www.miibeian.gov.cn/)

返回顶部